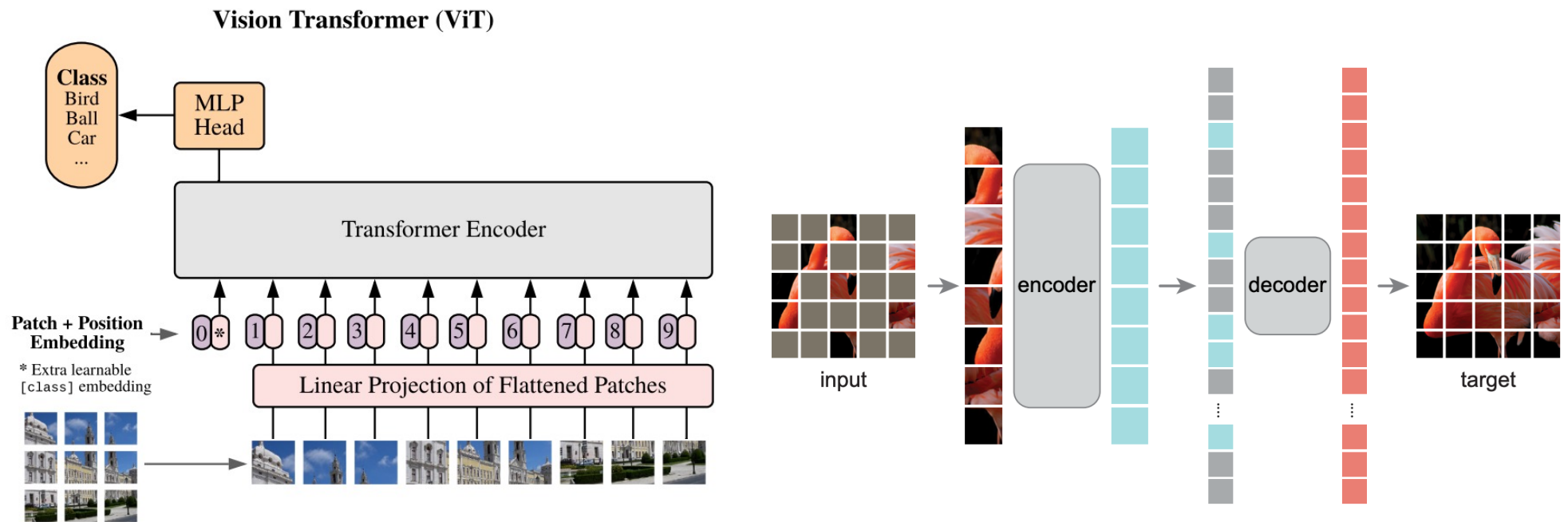
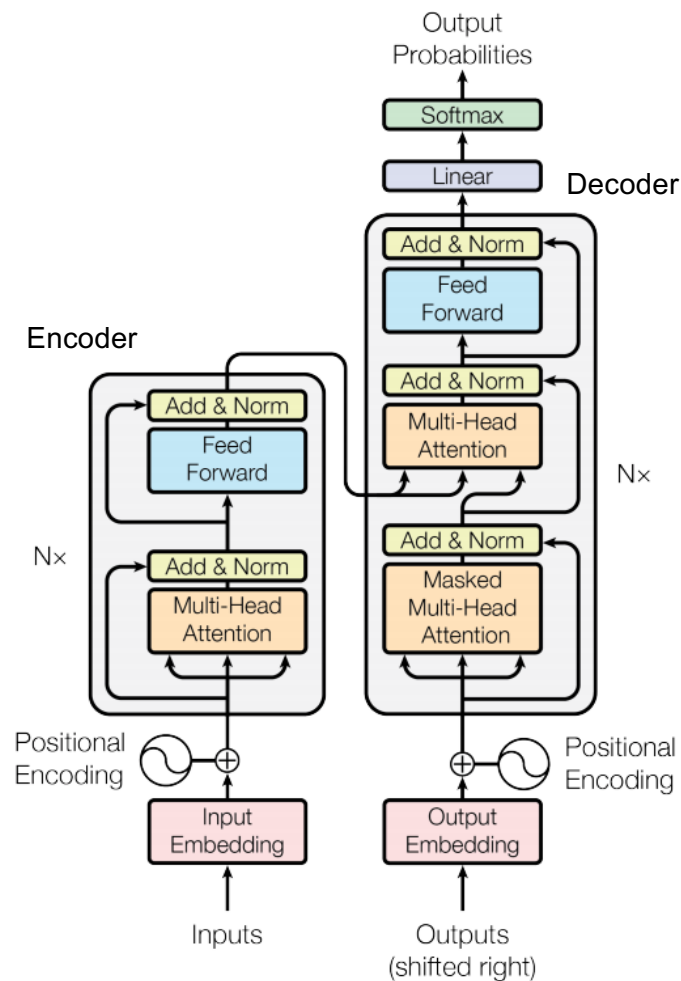


Transformers for image modeling



Last time: The original transformer

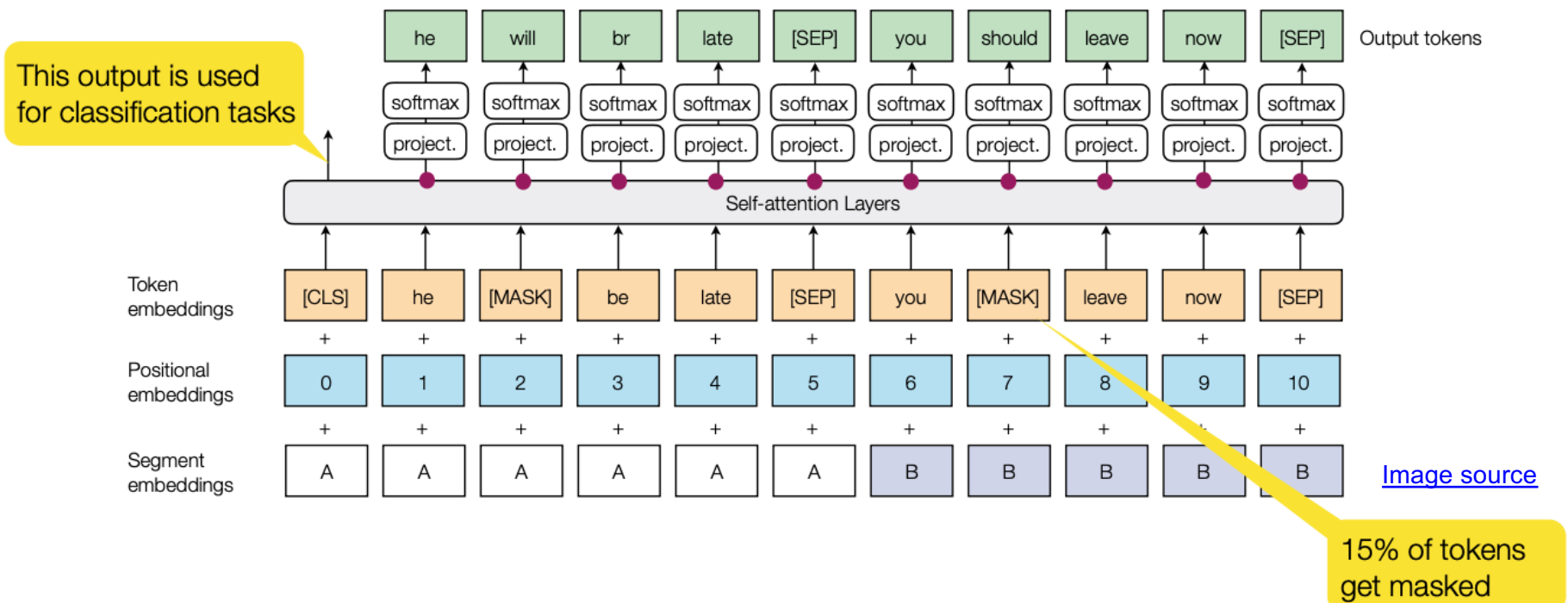


- Encoder-decoder architecture
- Attention is the only interaction between inputs!
- Three “flavors” of key-value-query attention
- Hyperparameters: embedding dimension, number of blocks, number of attention heads

A. Vaswani et al. [Attention is all you need](#). NeurIPS 2017

Last time: BERT

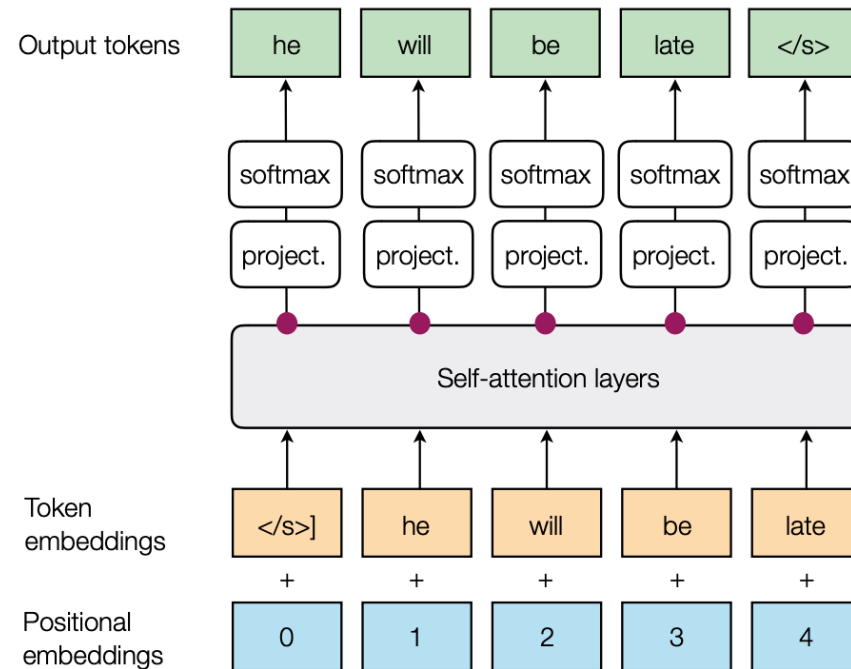
- Encoder-only architecture
- Pre-training task: masked language modeling



J. Devlin et al. [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#). EMNLP 2018

Last time: GPT

- Decoder-only architecture
- Pre-training task: next token prediction (causal language modeling)



A. Radford et al. [Improving language understanding by generative pre-training](#). 2018

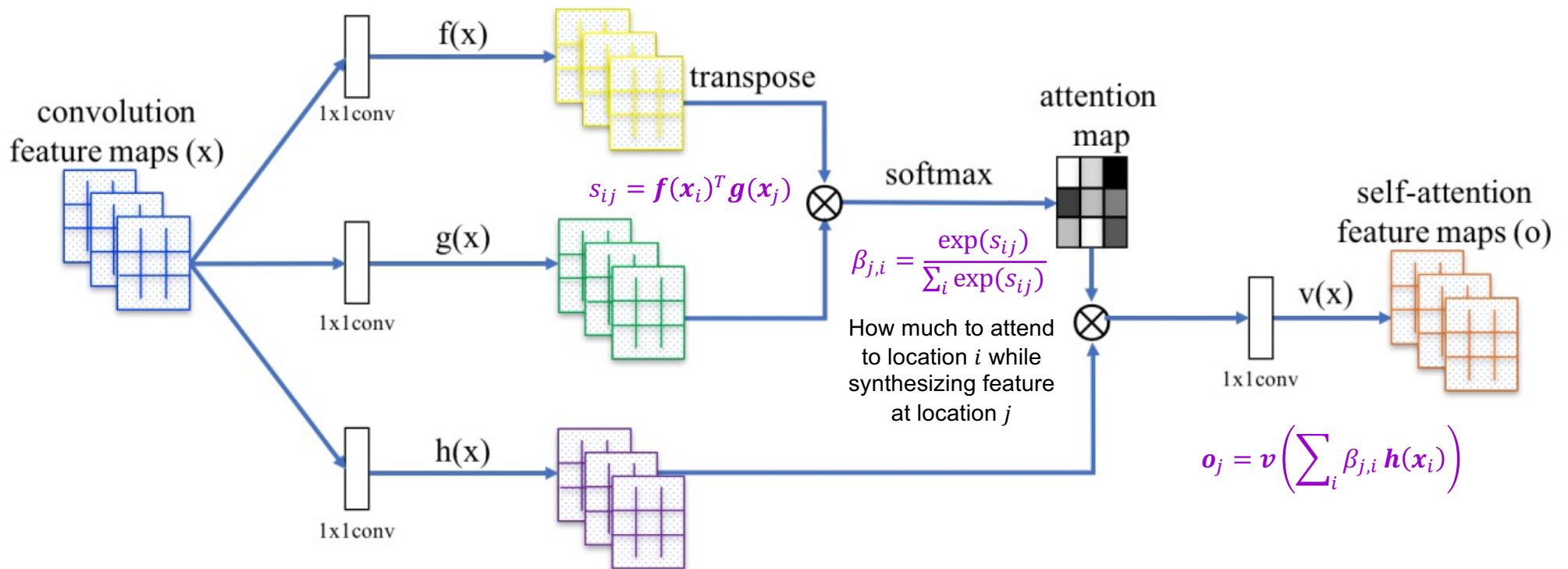
[Image source](#)

Outline

- Transformer backbones for images: ViT, SWIN
- Self-supervised pre-training: DINO, MAE
- Dense prediction with transformers: Mask2Former, SAM
- Alternatives to transformer architectures?

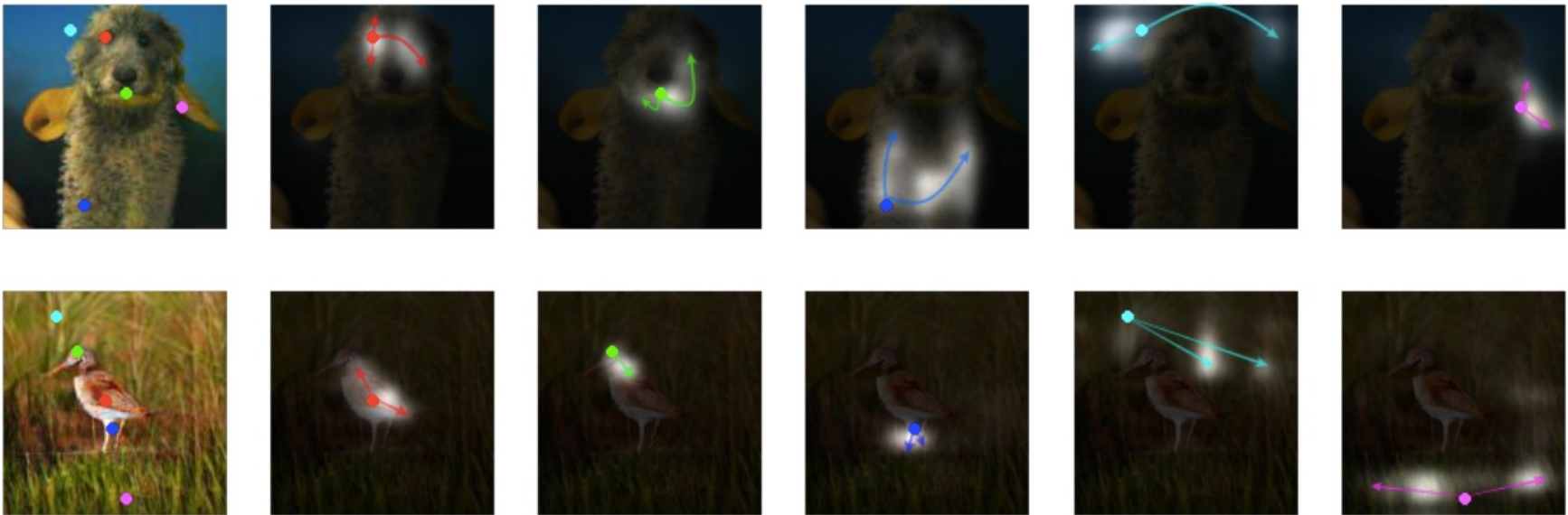
How can we use transformer models for vision?

Idea #1: Add self-attention layers to CNN architectures

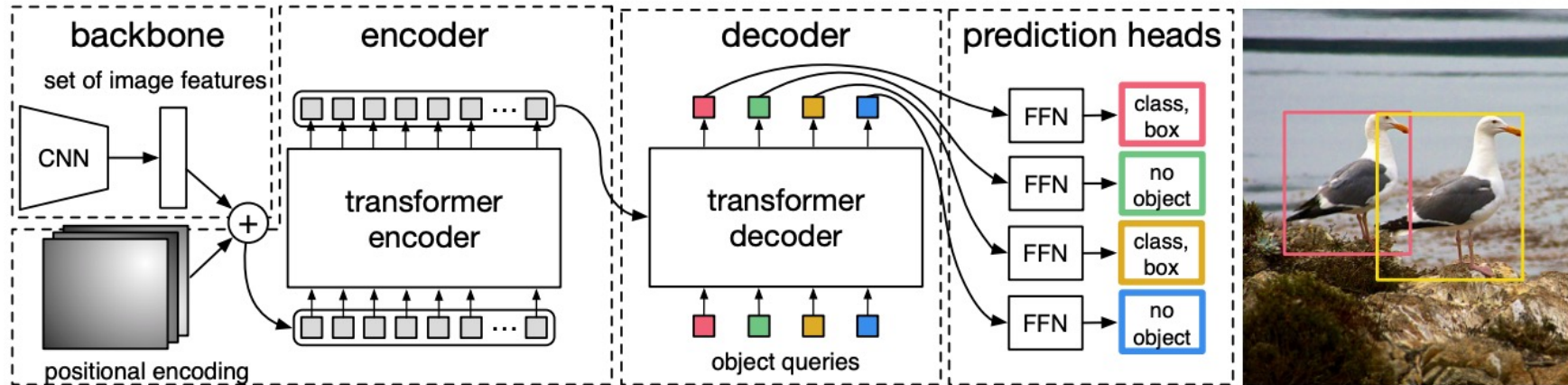
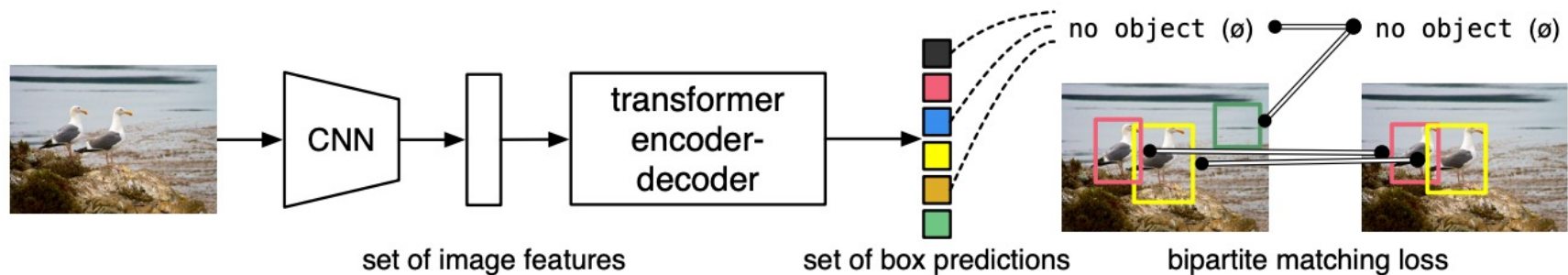


Idea #1: Add self-attention layers to CNN architectures

- Self-attention weights produce adaptive receptive fields that can better capture non-local structure

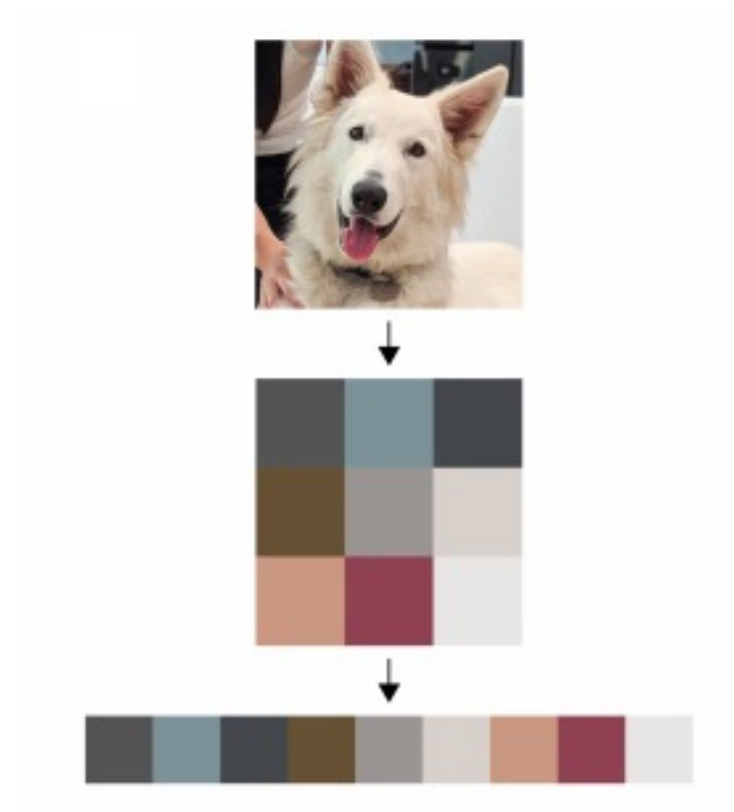


Idea #2: Transformer on top of a CNN backbone



N. Carion et al. [End-to-end object detection with transformers](#), ECCV 2020

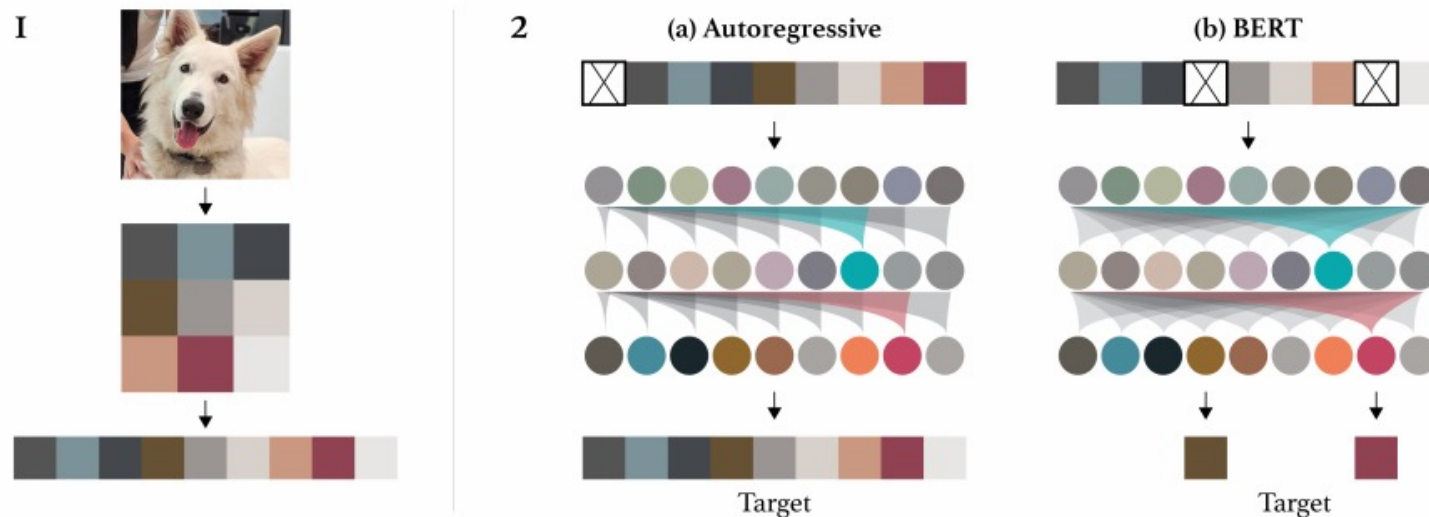
Idea #3: Transformer on top of pixels



M. Chen et al. [Generative pretraining from pixels](#). ICML 2020

Idea #3: Transformer on top of pixels

- ImageGPT: Image resolution up to 64x64, color values quantized to 512 levels (9 bits), dense attention
- Tried both autoregressive (GPT-style) and masked (BERT-style) pre-training



M. Chen et al. [Generative pretraining from pixels](#). ICML 2020

Idea #3: Transformer on top of pixels

- For transfer learning, average-pool encoded features across all positions

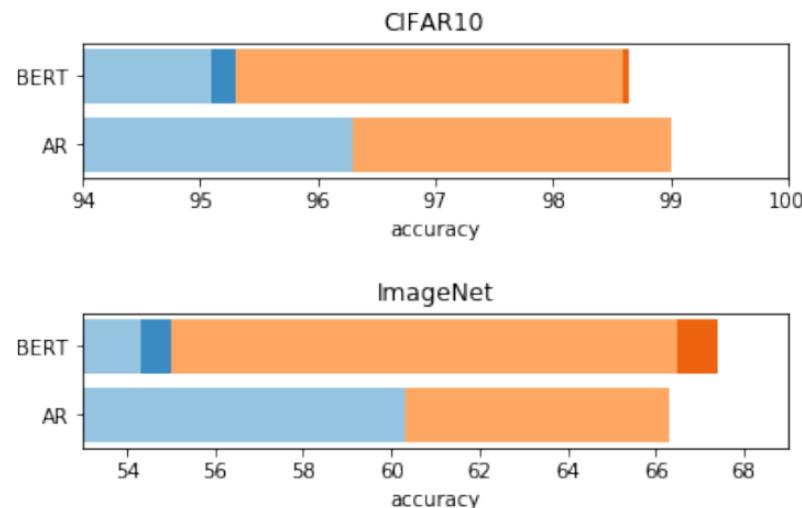


Figure 4. Comparison of auto-regressive pre-training with BERT pre-training using iGPT-L at an input resolution of $32^2 \times 3$. Blue bars display linear probe accuracy and orange bars display fine-tune accuracy. Bold colors show the performance boost from ensembling BERT masks. We see that auto-regressive models produce much better features than BERT models after pre-training, but BERT models catch up after fine-tuning.

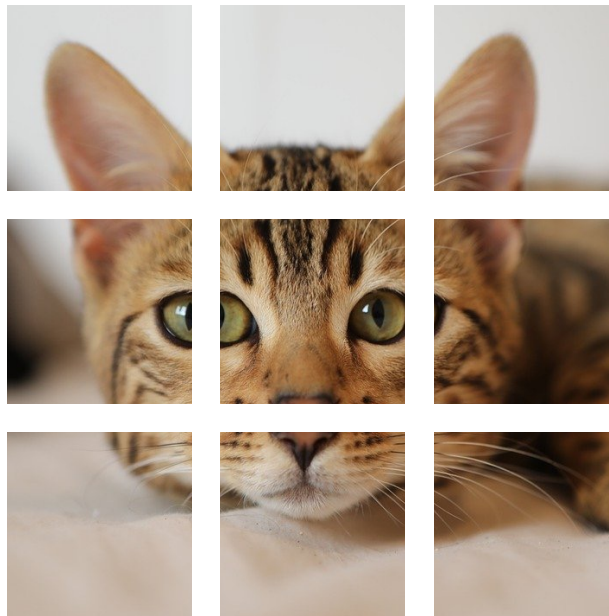
M. Chen et al. [Generative pretraining from pixels](#). ICML 2020

Idea #4: Transformer on top of patches



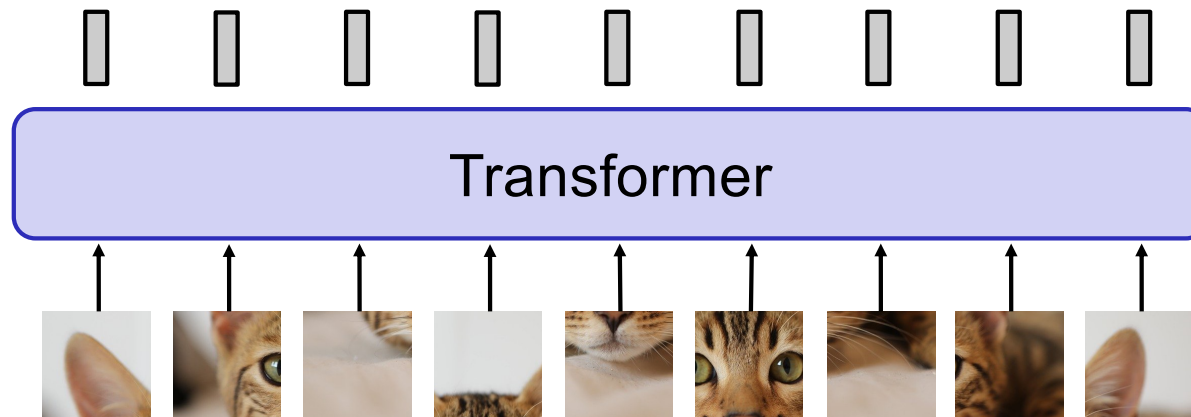
Source: [J. Johnson](#)

Idea #4: Transformer on top of patches



Source: [J. Johnson](#)

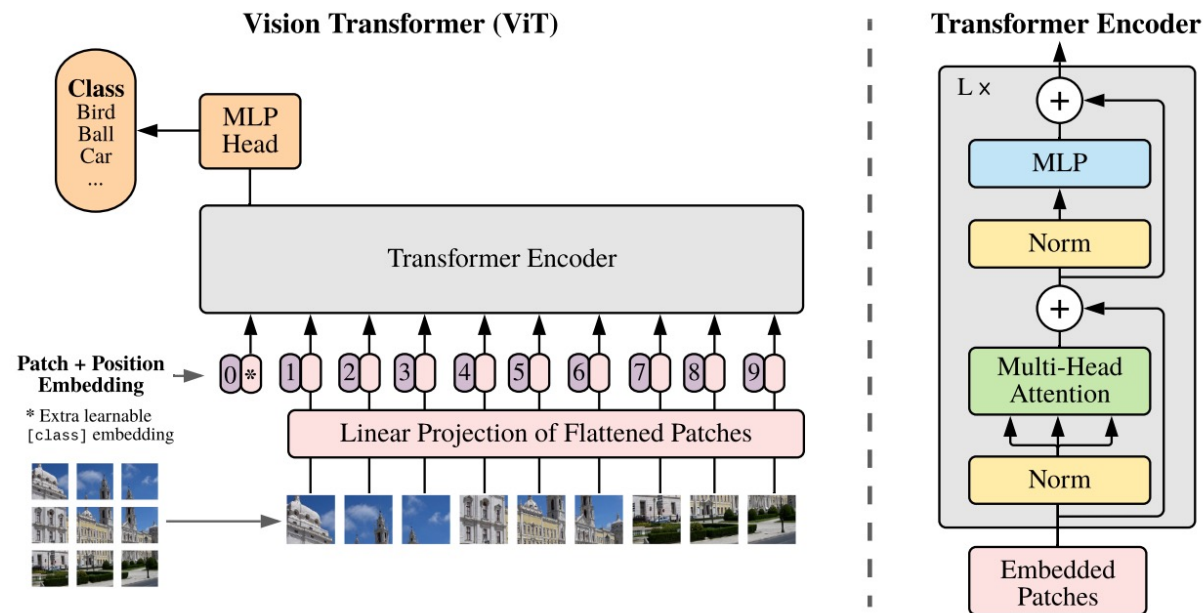
Idea #4: Transformer on top of patches



Source: [J. Johnson](#)

Vision transformer (ViT)

- Split an image into patches, feed linearly projected patches into standard transformer encoder
 - With patches of 14x14 pixels, you need $16 \times 16 = 256$ patches to represent 224x224 images



A. Dosovitskiy et al. [An image is worth 16x16 words: Transformers for image recognition at scale](#). ICLR 2021

Vision transformer (ViT)

- Split an image into patches, feed linearly projected patches into standard transformer encoder
 - With patches of 14x14 pixels, you need $16 \times 16 = 256$ patches to represent 224x224 images

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Table 1: Details of Vision Transformer model variants.

Vision transformer (ViT)

- Trained in a supervised fashion, fine-tuned on ImageNet

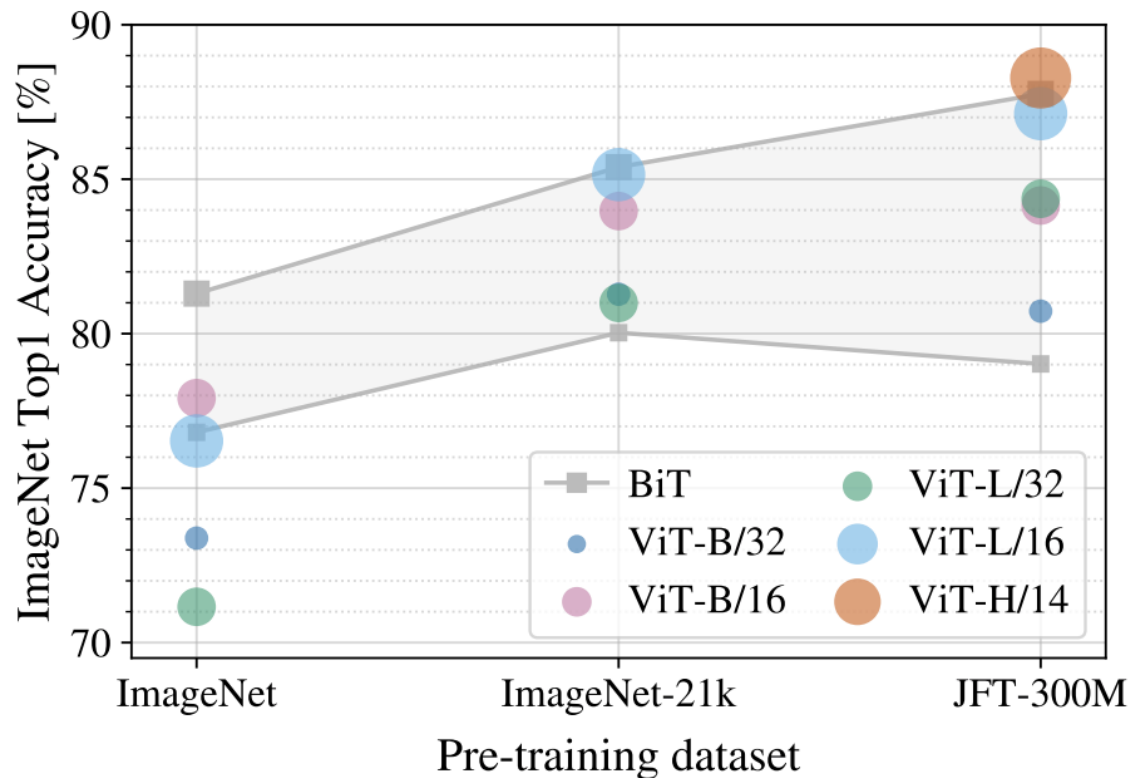


Figure 3: Transfer to ImageNet. While large ViT models perform worse than BiT ResNets (shaded area) when pre-trained on small datasets, they shine when pre-trained on larger datasets. Similarly, larger ViT variants overtake smaller ones as the dataset grows.

BiT: [Big Transfer](#) (ResNet)

ViT: Vision Transformer (Base/Large/Huge, patch size of 14x14, 16x16, or 32x32)

[Internal Google dataset](#) (not public)

Vision transformer (ViT)

- Transformers are claimed to be more computationally efficient than CNNs or hybrid architectures

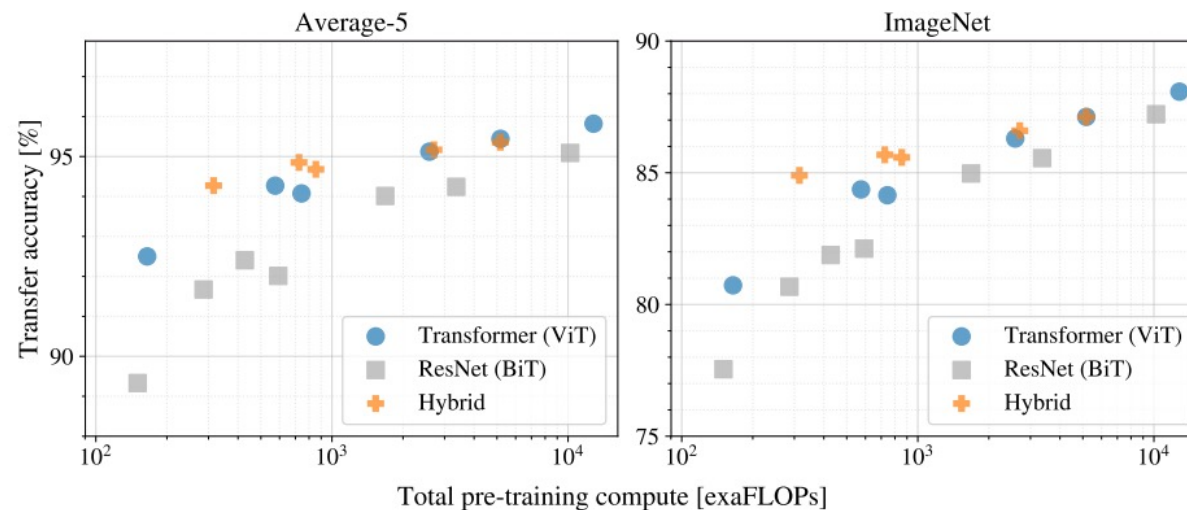
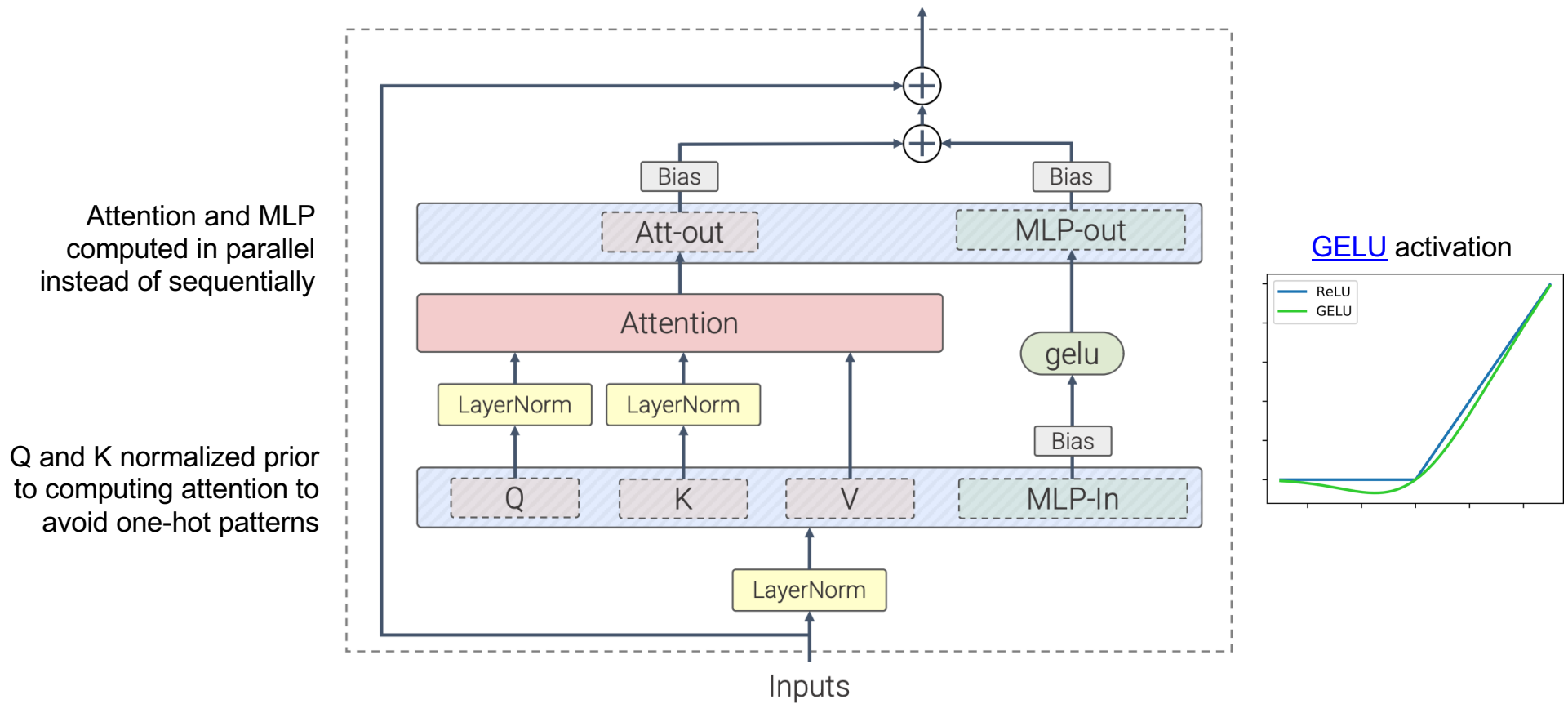


Figure 5: Performance versus cost for different architectures: Vision Transformers, ResNets, and hybrids. Vision Transformers generally outperform ResNets with the same computational budget. Hybrids improve upon pure Transformers for smaller model sizes, but the gap vanishes for larger models.

A. Dosovitskiy et al. [An image is worth 16x16 words: Transformers for image recognition at scale](#). ICLR 2021

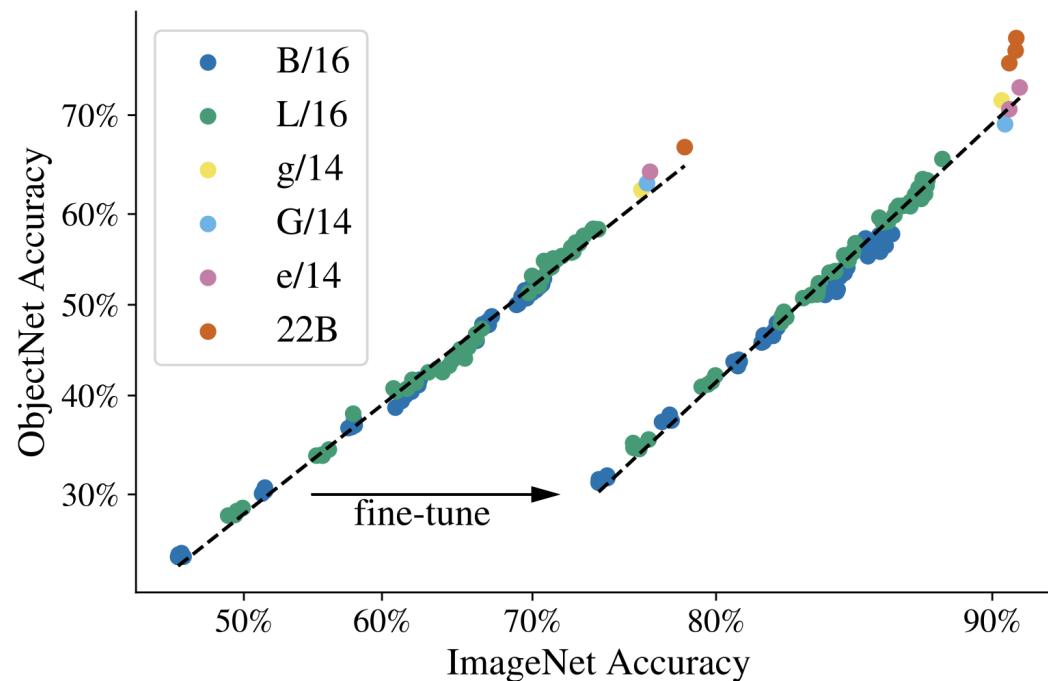
ViT-22B



M. Dehghani et al. [Scaling Vision Transformers to 22 Billion Parameters](#). arXiv 2023

ViT-22B

- Trained on an extended version of JFT dataset with 4B images “semi-automatically” annotated with 30K labels



M. Dehghani et al. [Scaling Vision Transformers to 22 Billion Parameters](#). arXiv 2023

Hierarchical transformer: Swin

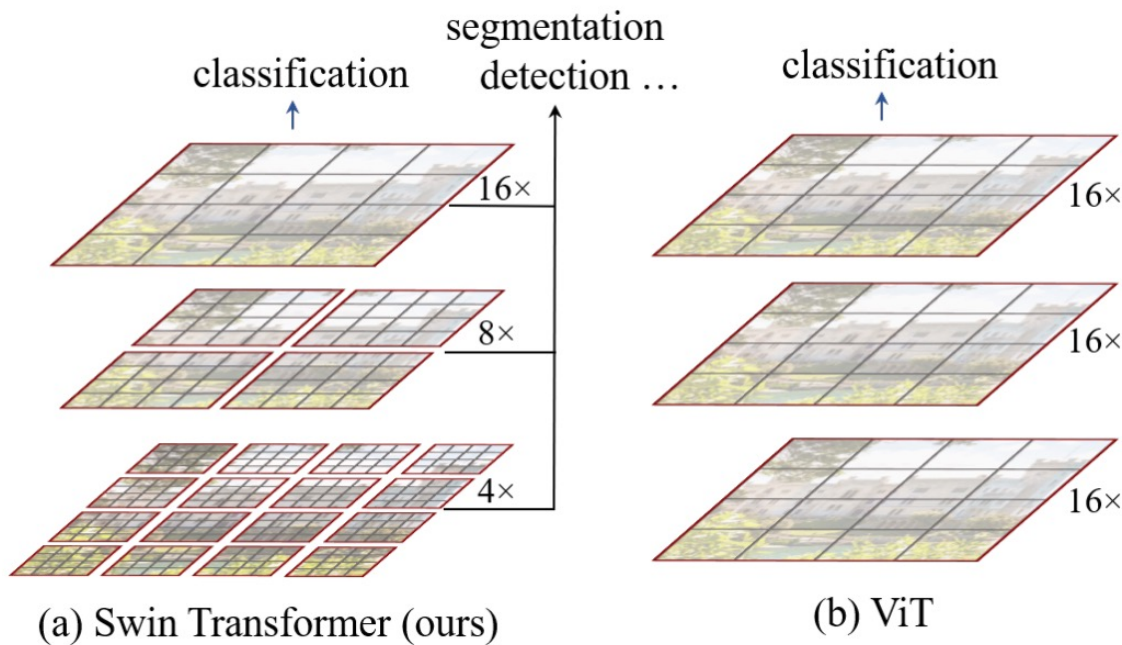


Figure 1. (a) The proposed Swin Transformer builds hierarchical feature maps by merging image patches (shown in gray) in deeper layers and has linear computation complexity to input image size due to computation of self-attention only within each local window (shown in red). It can thus serve as a general-purpose backbone for both image classification and dense recognition tasks. (b) In contrast, previous vision Transformers [19] produce feature maps of a single low resolution and have quadratic computation complexity to input image size due to computation of self-attention globally.

Hierarchical transformer: Swin

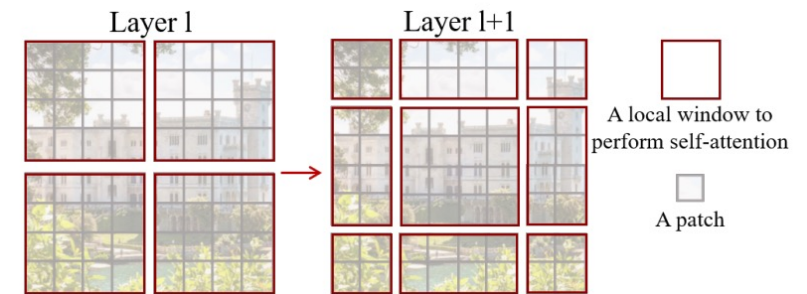
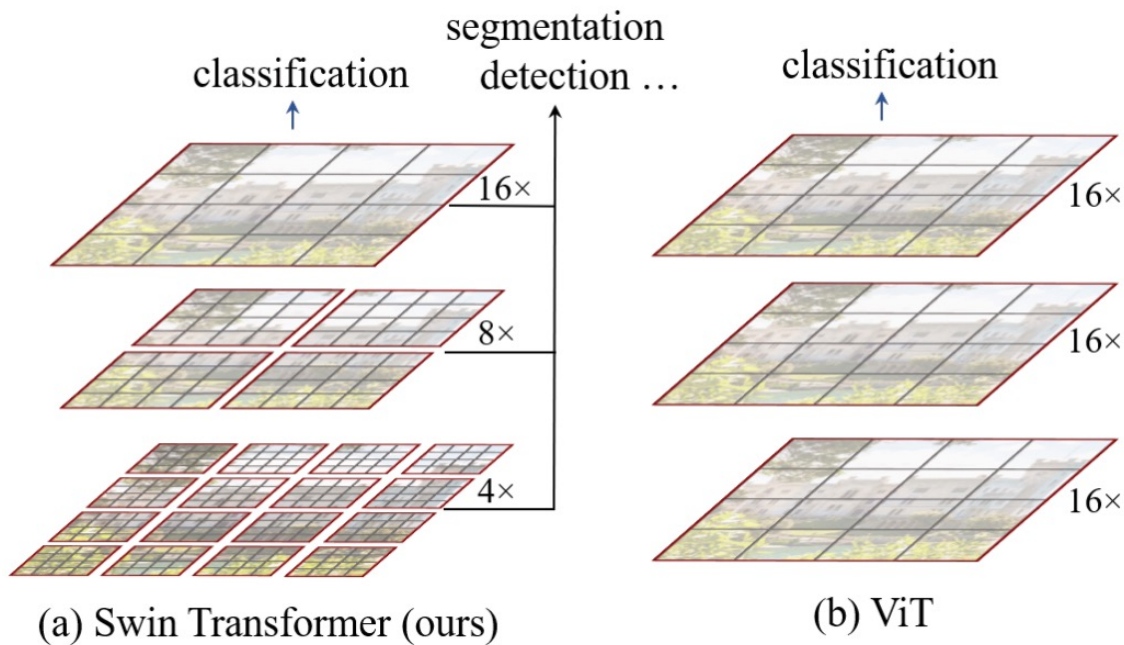


Figure 2. An illustration of the *shifted window* approach for computing self-attention in the proposed Swin Transformer architecture. In layer l (left), a regular window partitioning scheme is adopted, and self-attention is computed within each window. In the next layer $l + 1$ (right), the window partitioning is shifted, resulting in new windows. The self-attention computation in the new windows crosses the boundaries of the previous windows in layer l , providing connections among them.

Hierarchical transformer: Swin

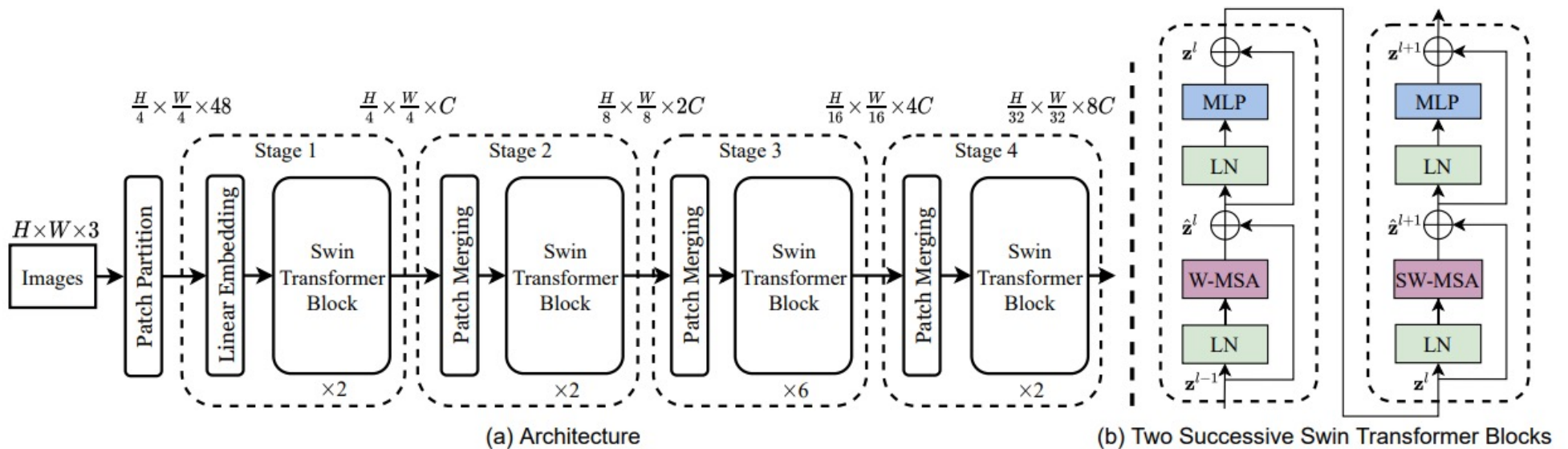


Figure 3. (a) The architecture of a Swin Transformer (Swin-T); (b) two successive Swin Transformer Blocks (notation presented with Eq. (3)). W-MSA and SW-MSA are multi-head self attention modules with regular and shifted windowing configurations, respectively.

Swin results

(a) Regular ImageNet-1K trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [44]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [44]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [44]	224 ²	84M	16.0G	334.7	82.9
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [57]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [57]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [57]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.5
Swin-B	384 ²	88M	47.0G	84.7	84.5

(b) ImageNet-22K pre-trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [34]	384 ²	388M	204.6G	-	84.4
R-152x4 [34]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.4
Swin-L	384 ²	197M	103.9G	42.1	87.3

Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [62] and a V100 GPU, following [57].

COCO detection and segmentation

(a) Various frameworks

Method	Backbone	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	#param.	FLOPs	FPS
Cascade	R-50	46.3	64.3	50.5	82M	739G	18.0
Mask R-CNN	Swin-T	50.5	69.3	54.9	86M	745G	15.3
ATSS	R-50	43.5	61.9	47.0	32M	205G	28.3
	Swin-T	47.2	66.5	51.3	36M	215G	22.3
RepPointsV2	R-50	46.5	64.6	50.3	42M	274G	13.6
	Swin-T	50.0	68.5	54.2	45M	283G	12.0
Sparse R-CNN	R-50	44.5	63.4	48.2	106M	166G	21.0
	Swin-T	47.9	67.3	52.3	110M	172G	18.4

(b) Various backbones w. Cascade Mask R-CNN

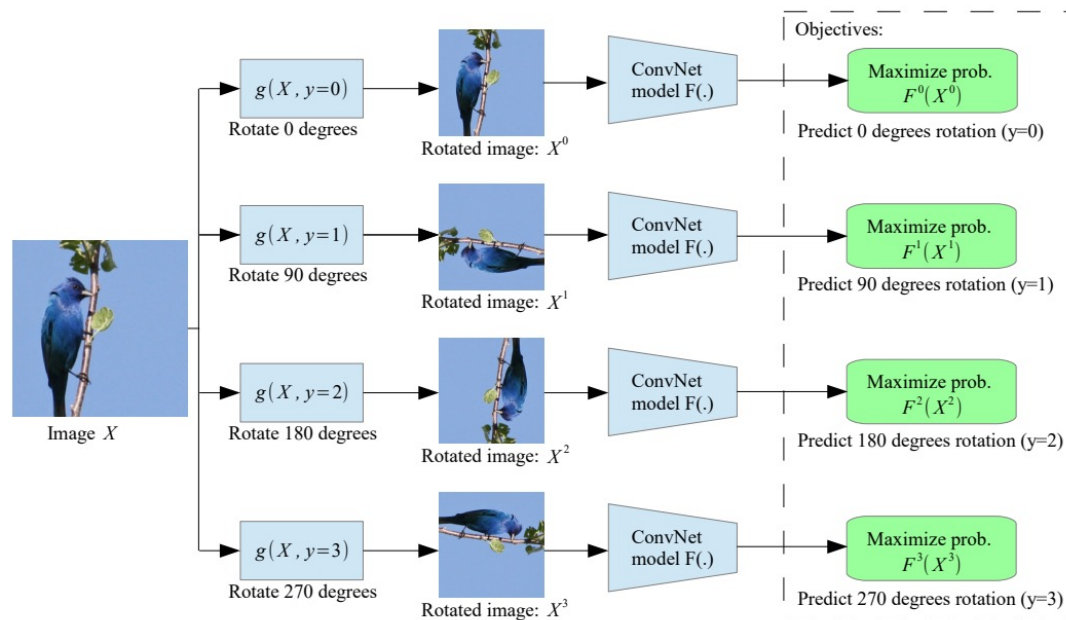
	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	AP ^{mask}	AP ^{mask} ₅₀	AP ^{mask} ₇₅	#param	FLOPs	FPS
DeiT-S [†]	48.0	67.2	51.7	41.4	64.2	44.3	80M	889G	10.4
R50	46.3	64.3	50.5	40.1	61.7	43.4	82M	739G	18.0
Swin-T	50.5	69.3	54.9	43.7	66.6	47.1	86M	745G	15.3
X101-32	48.1	66.5	52.4	41.6	63.9	45.2	101M	819G	12.8
Swin-S	51.8	70.4	56.3	44.7	67.9	48.5	107M	838G	12.0
X101-64	48.3	66.4	52.3	41.7	64.0	45.1	140M	972G	10.4
Swin-B	51.9	70.9	56.5	45.0	68.4	48.7	145M	982G	11.6

Outline

- Transformer backbones for images: ViT, SWIN
- Self-supervised pre-training: DINO, MAE

Self-supervised pre-training

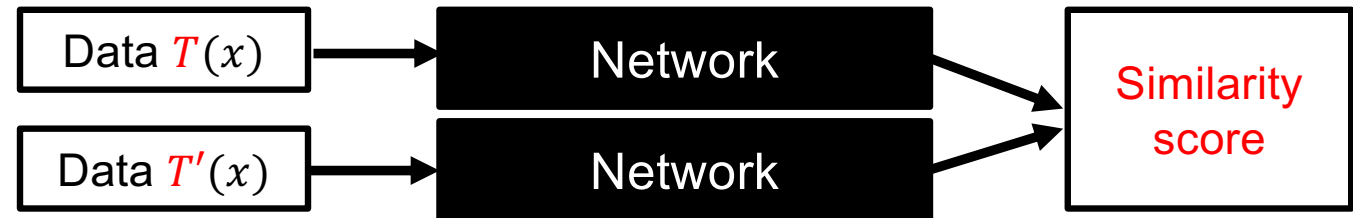
Transformation
prediction



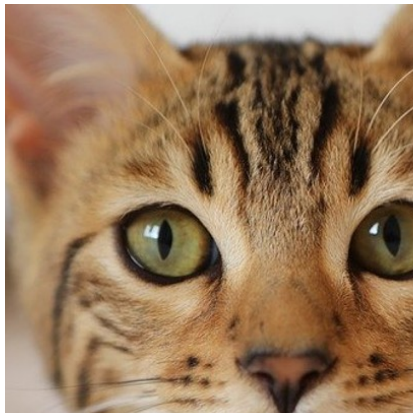
S. Gidaris et al. [Unsupervised representation learning by predicting image rotations](#). ICLR 2018

Self-supervised pre-training

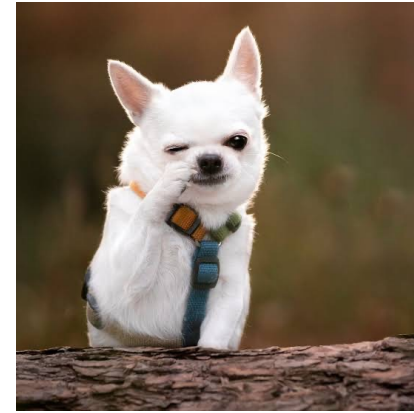
“Siamese”
methods



$T(x)$



$T'(x)$



Representations should
be similar

Representations should be
dissimilar

DINO: Self-distillation with no labels

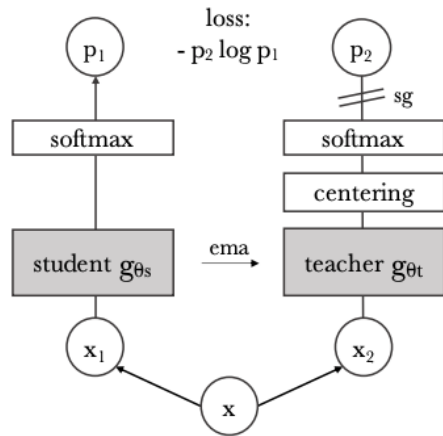


Figure 2: **Self-distillation with no labels.** We illustrate DINO in the case of one single pair of views (x_1, x_2) for simplicity. The model passes two different random transformations of an input image to the student and teacher networks. Both networks have the same architecture but different parameters. The output of the teacher network is centered with a mean computed over the batch. Each network outputs a K dimensional feature that is normalized with a temperature softmax over the feature dimension. Their similarity is then measured with a cross-entropy loss. We apply a stop-gradient (sg) operator on the teacher to propagate gradients only through the student. The teacher parameters are updated with an exponential moving average (ema) of the student parameters.

- Tricks (to avoid collapse):
 - **Student-teacher training** with **distillation** loss (cross-entropy on top of softmax outputs)
 - Apply **centering** to teacher output followed by **sharpening** (softmax with low temperature)
 - **Stop-gradient** (sg): do not update parameters of the teacher branch with back-propagation
 - Instead, update teacher parameters with **exponential moving average** (ema) of student parameters

DINO: Self-distillation with no labels

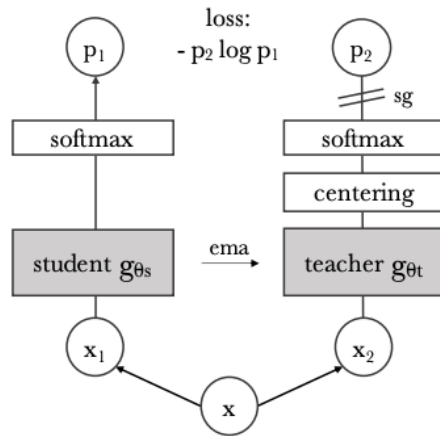


Figure 2: **Self-distillation with no labels.** We illustrate DINO in the case of one single pair of views (x_1, x_2) for simplicity. The model passes two different random transformations of an input image to the student and teacher networks. Both networks have the same architecture but different parameters. The output of the teacher network is centered with a mean computed over the batch. Each network outputs a K dimensional feature that is normalized with a temperature softmax over the feature dimension. Their similarity is then measured with a cross-entropy loss. We apply a stop-gradient (sg) operator on the teacher to propagate gradients only through the student. The teacher parameters are updated with an exponential moving average (ema) of the student parameters.

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

M. Caron et al. [Emerging Properties in Self-Supervised Vision Transformers](#). ICCV 2021

DINO: Self-distillation with no labels

Method	Arch.	Param.	im/s	Linear	k -NN
Supervised	RN50	23	1237	79.3	79.3
SCLR [11]	RN50	23	1237	69.1	60.7
MoCov2 [13]	RN50	23	1237	71.1	61.9
InfoMin [54]	RN50	23	1237	73.0	65.3
BarlowT [66]	RN50	23	1237	73.2	66.0
OBoW [21]	RN50	23	1237	73.8	61.9
BYOL [23]	RN50	23	1237	74.4	64.8
DCv2 [9]	RN50	23	1237	75.2	67.1
SwAV [9]	RN50	23	1237	75.3	65.7
DINO	RN50	23	1237	75.3	67.5
Supervised	ViT-S	21	1007	79.8	79.8
BYOL* [23]	ViT-S	21	1007	71.4	66.6
MoCov2* [13]	ViT-S	21	1007	72.7	64.4
SwAV* [9]	ViT-S	21	1007	73.5	66.3
DINO	ViT-S	21	1007	77.0	74.5

Table 2: **Linear and k -NN classification on ImageNet.** We report top-1 accuracy for linear and k -NN evaluations on the validation set of ImageNet for different self-supervised methods. We focus on ResNet-50 and ViT-small architectures, but also report the best results obtained across architectures. * are run by us. We run the k -NN evaluation for models with official released weights. The throughput (im/s) is calculated on a NVIDIA V100 GPU with 128 samples per forward. Parameters (M) are of the feature extractor.

M. Caron et al. [Emerging Properties in Self-Supervised Vision Transformers](#). ICCV 2021

DINO: Self-distillation with no labels

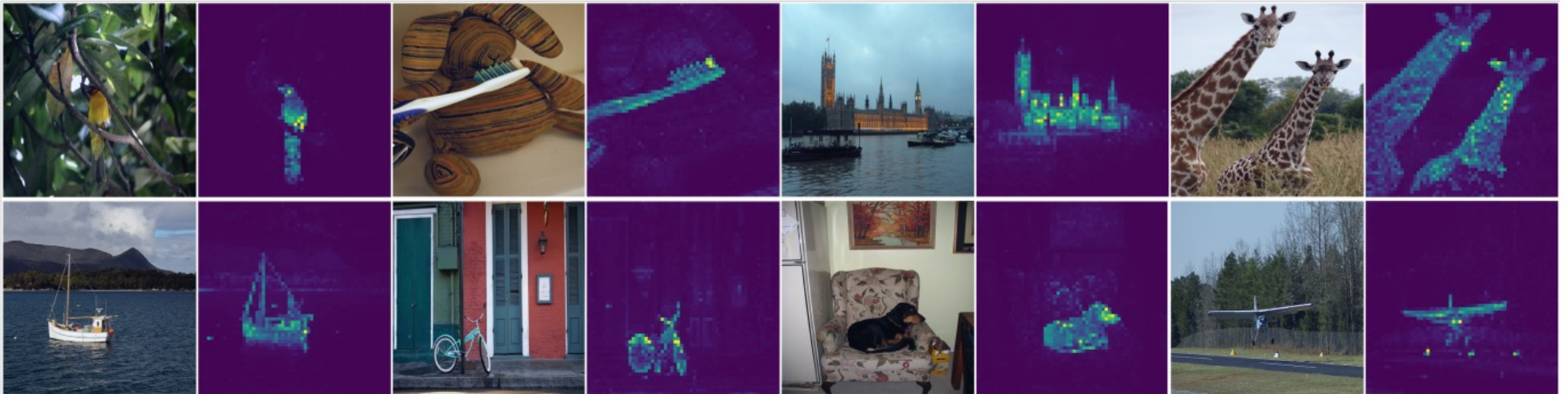


Figure 1: **Self-attention from a Vision Transformer with 8×8 patches trained with no supervision.** We look at the self-attention of the [CLS] token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

M. Caron et al. [Emerging Properties in Self-Supervised Vision Transformers](#). ICCV 2021

DINO: Self-distillation with no labels

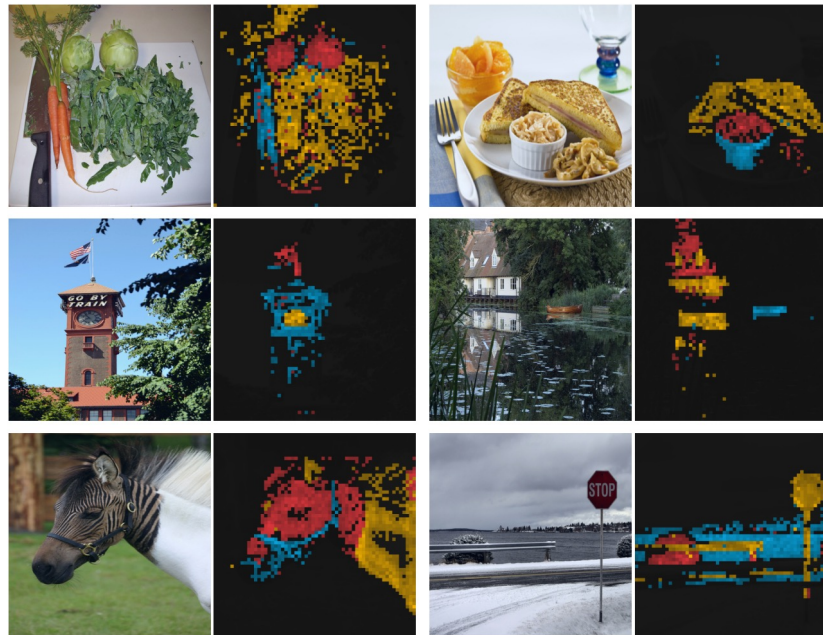


Figure 3: **Attention maps from multiple heads.** We consider the heads from the last layer of a ViT-S/8 trained with DINO and display the self-attention for [CLS] token query. Different heads, materialized by different colors, focus on different locations that represents different objects or parts (more examples in Appendix).

M. Caron et al. [Emerging Properties in Self-Supervised Vision Transformers](#). ICCV 2021

DINOv2

The recent breakthroughs in natural language processing for model pretraining on large quantities of data have opened the way for similar foundation models in computer vision. These models could greatly simplify the use of images in any system by producing general-purpose visual features, i.e., features that work across image distributions and tasks without finetuning. This work shows that existing pretraining methods, especially self-supervised methods, can produce such features if trained on enough curated data from diverse sources. We revisit existing approaches and combine different techniques to scale our pretraining in terms of data and model size. Most of the technical contributions aim at accelerating and stabilizing the training at scale. In terms of data, we propose an automatic pipeline to build a dedicated, diverse, and curated image dataset instead of uncurated data, as typically done in the self-supervised literature. In terms of models, we train a ViT model (Dosovitskiy et al., 2021) with 1B parameters and distill it into a series of smaller models that surpass the best available general-purpose features, OpenCLIP (Ilharco et al., 2021) on most of the benchmarks at image and pixel levels.

M. Oquab et al. [DINOv2: Learning Robust Visual Features without Supervision](#). TMLR 2024

DINOv2

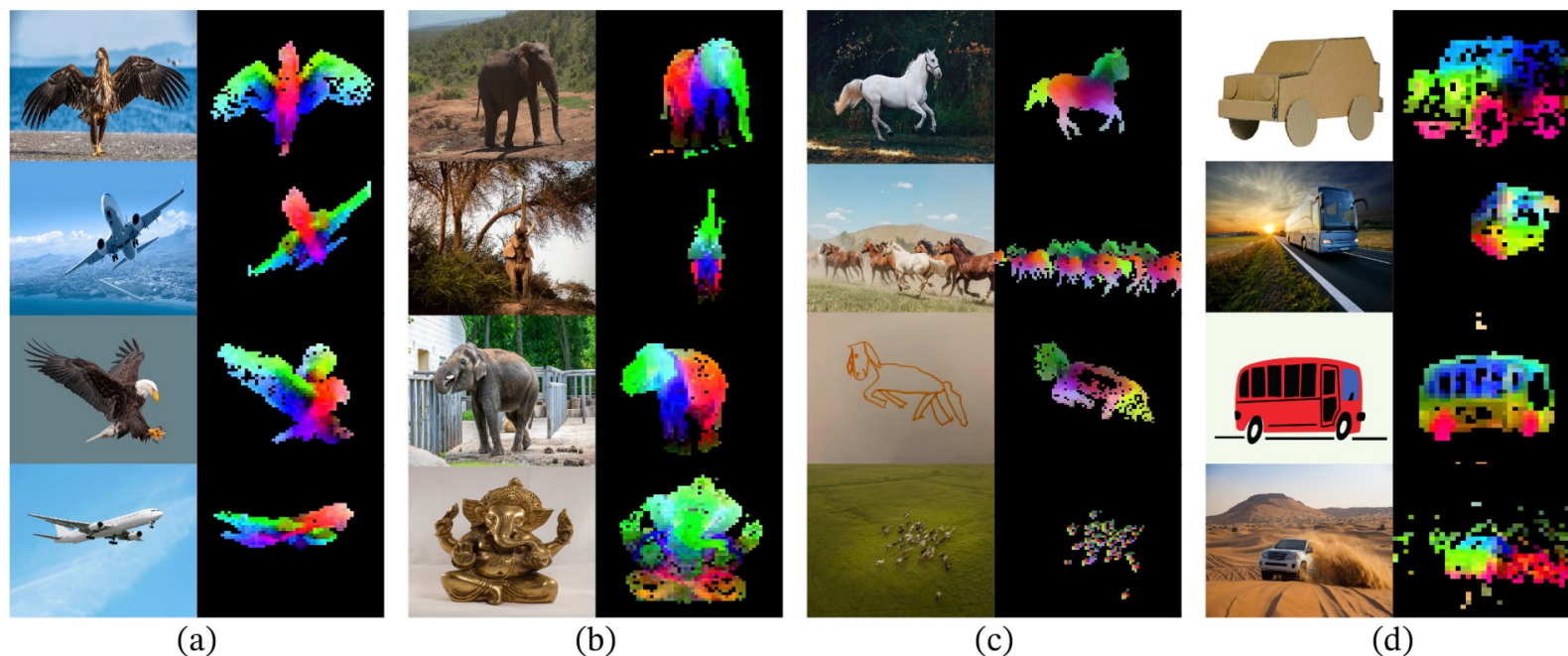


Figure 1: **Visualization of the first PCA components.** We compute a PCA between the patches of the images from the same column (a, b, c and d) and show their first 3 components. Each component is matched to a different color channel. Same parts are matched between related images despite changes of pose, style or even objects. Background is removed by thresholding the first PCA component.

M. Oquab et al. [DINOv2: Learning Robust Visual Features without Supervision](#). TMLR 2024

DINOv2

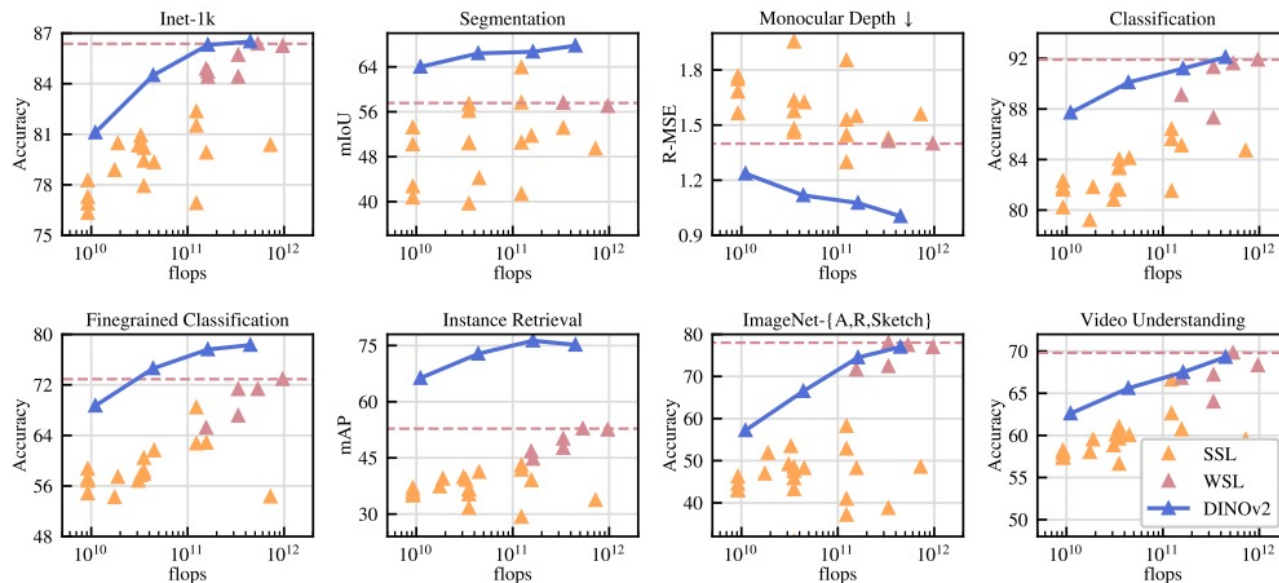
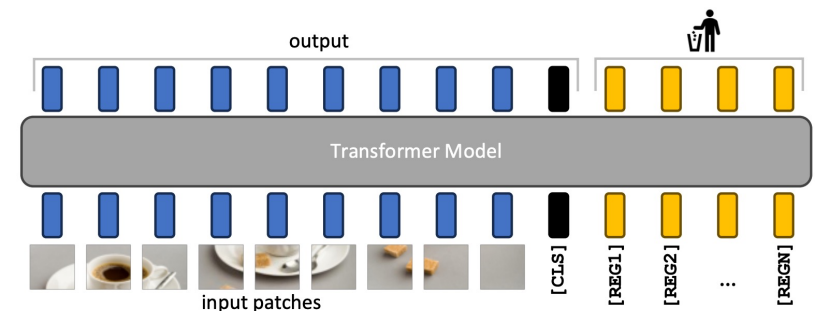
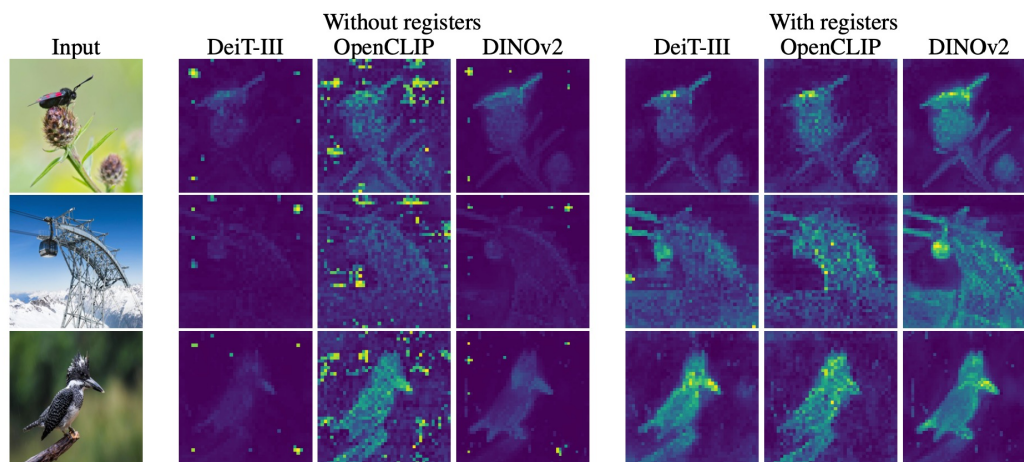


Figure 2: **Evolution of performance when scaling in parameters.** We show performance on eight types of vision tasks, as presented in Sec. 7, and average metrics with each type. Features are extracted from our self-supervised encoders, DINOv2 (dark blue), and we compare them with self-supervised methods (pale orange), as well as weakly-supervised methods (dark pink). We report the best-performing weakly-supervised model’s performance as a dashed horizontal line. Our family of models drastically improves over the previous state of the art in self-supervised learning and reaches performance comparable with weakly-supervised features. See Sec. 7 for a detailed analysis.

Refinement of DINOv2: ViT with registers

- “We make the following hypothesis: **large, sufficiently trained models learn to recognize redundant tokens, and to use them as places to store, process and retrieve global information...** While this behavior is not bad in itself, the fact that it happens inside the patch tokens is undesirable. Indeed, it leads the model to discard local patch information, possibly incurring decreased performance on dense prediction tasks. We therefore propose a simple fix to this issue: **we explicitly add new tokens to the sequence, that the model can learn to use as registers.**”



Refinement of DINOv2: ViT with registers

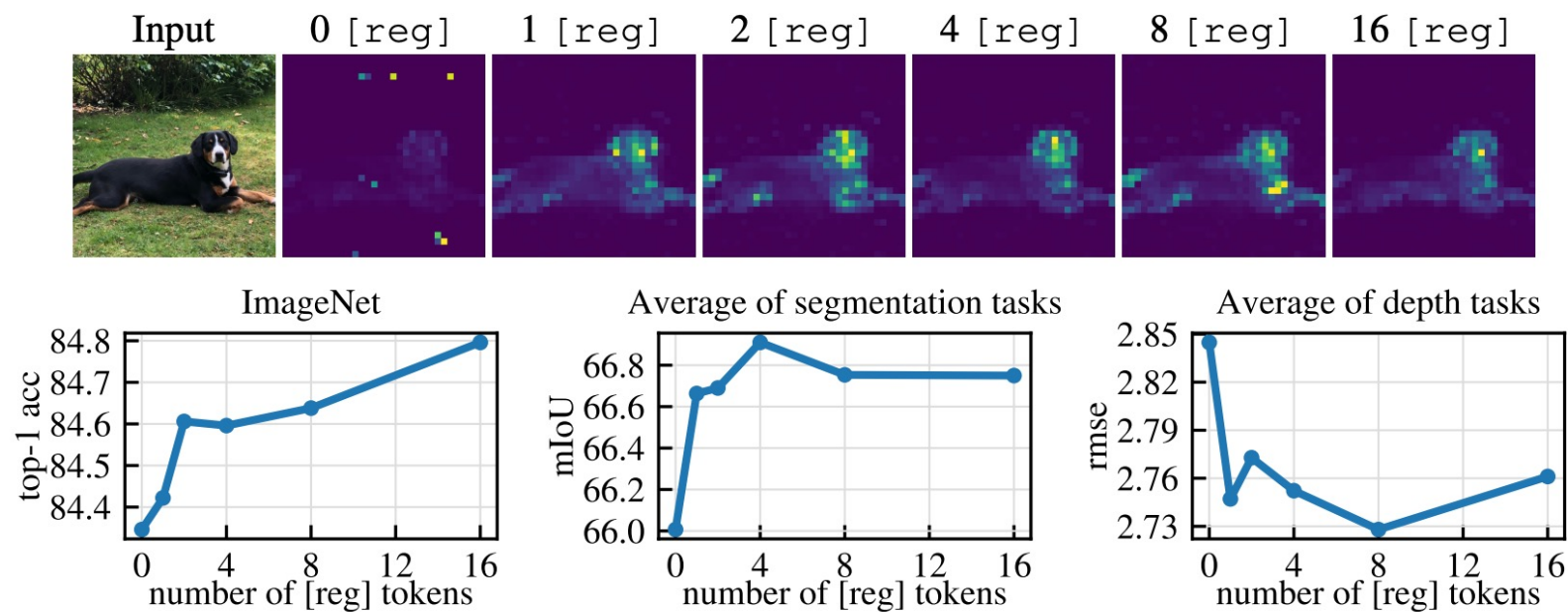


Figure 8: Ablation of the the number of register tokens used with a DINOv2 model. **(top)**: qualitative visualization of artifacts appearing as a function of number of registers. **(bottom)**: performance on three tasks (ImageNet, ADE-20k and NYUd) as a function of number of registers used. While one register is sufficient to remove artefacts, using more leads to improved downstream performance.

Refinement of DINOv2: ViT with registers

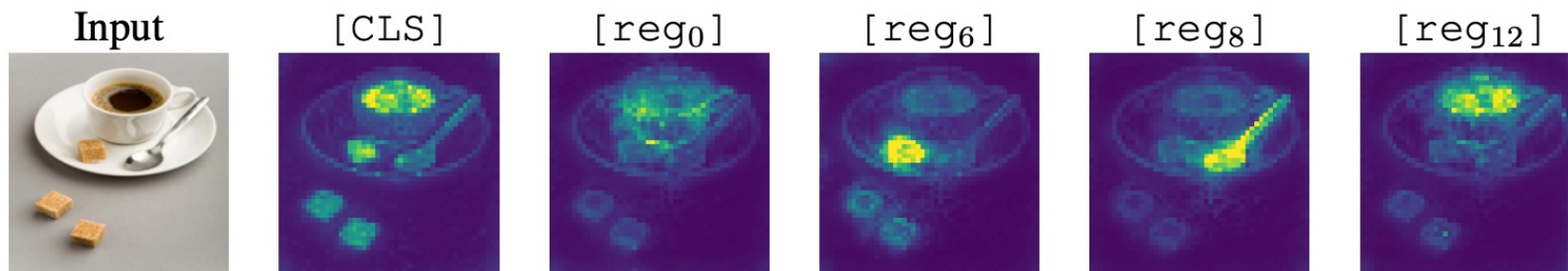
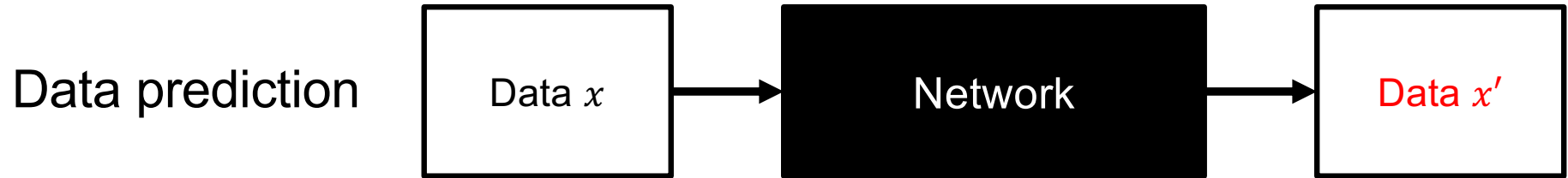


Figure 9: Comparison of the attention maps of the [CLS] and register tokens. Register tokens sometimes attend to different parts of the feature map, similarly to slot attention (Locatello et al., 2020). This behaviour was never required from the model, and emerged naturally from training.

Self-supervised pre-training



Masked autoencoders

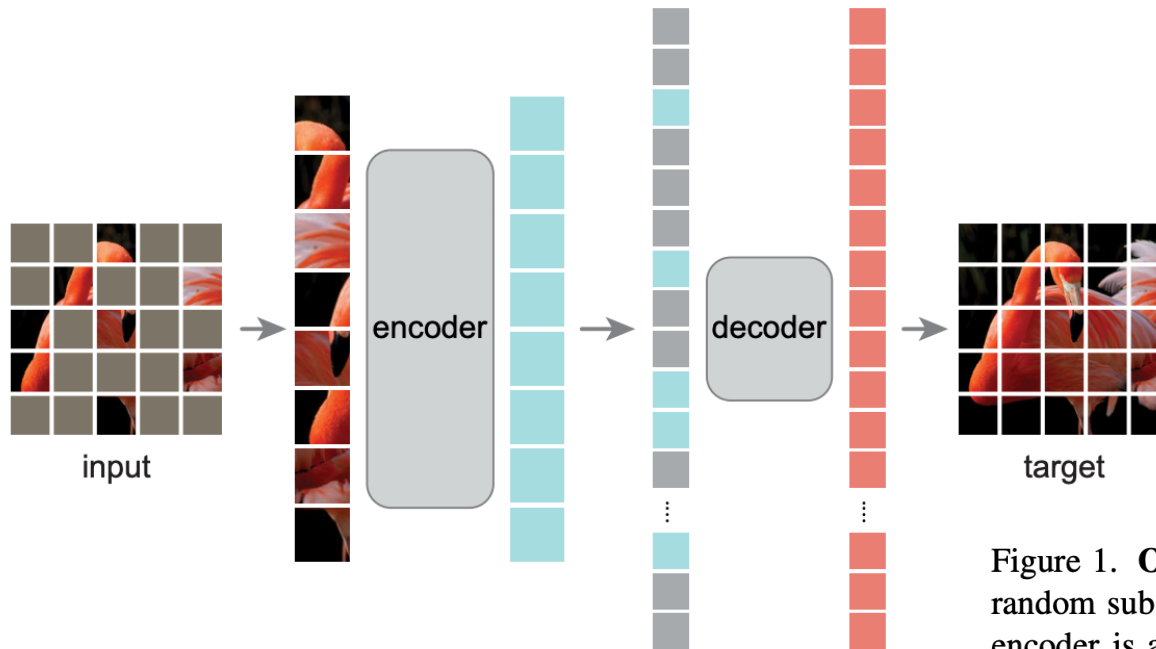


Figure 1. **Our MAE architecture.** During pre-training, a large random subset of image patches (*e.g.*, 75%) is masked out. The encoder is applied to the small subset of *visible patches*. Mask tokens are introduced *after* the encoder, and the full set of encoded patches and mask tokens is processed by a small decoder that reconstructs the original image in pixels. After pre-training, the decoder is discarded and the encoder is applied to uncorrupted images to produce representations for recognition tasks.

K. He et al. [Masked autoencoders are scalable vision learners](#). CVPR 2022

Masked autoencoders

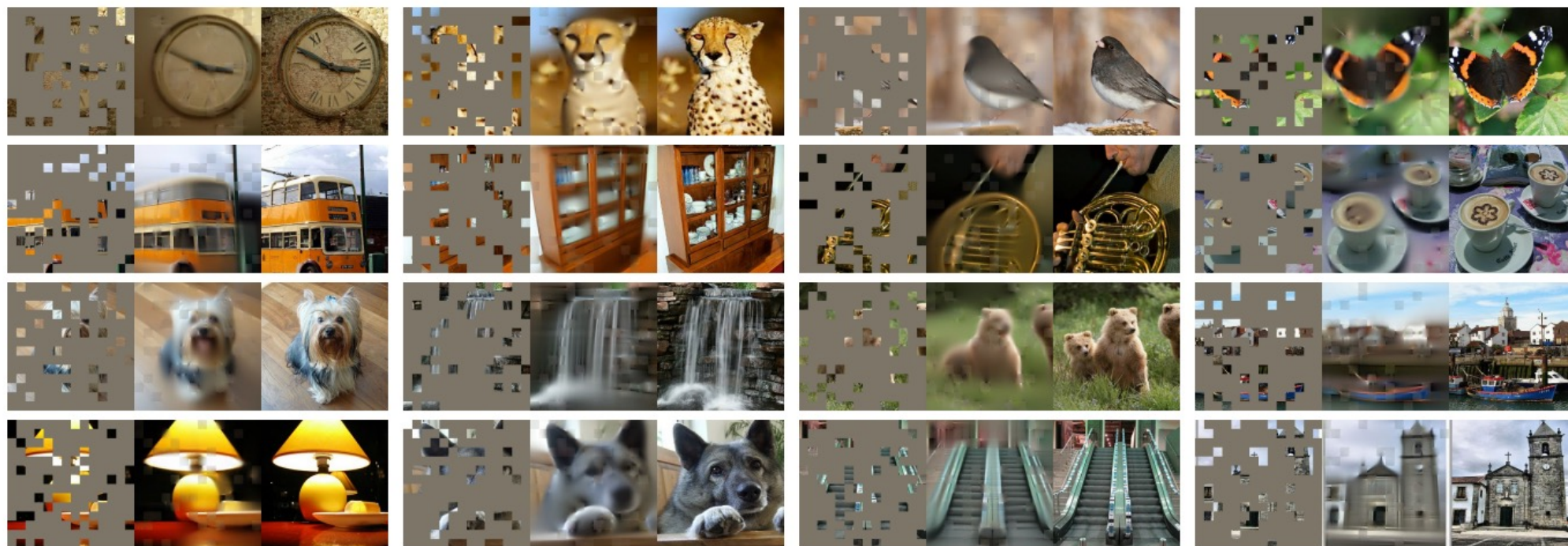


Figure 2. Example results on ImageNet *validation* images. For each triplet, we show the masked image (left), our MAE reconstruction[†] (middle), and the ground-truth (right). The masking ratio is 80%, leaving only 39 out of 196 patches. More examples are in the appendix.
[†]As no loss is computed on visible patches, the model output on visible patches is qualitatively worse. One can simply overlay the output with the visible patches to improve visual quality. We intentionally opt not to do this, so we can more comprehensively demonstrate the method’s behavior.

K. He et al. [Masked autoencoders are scalable vision learners](#). CVPR 2022

Masked autoencoders: Results

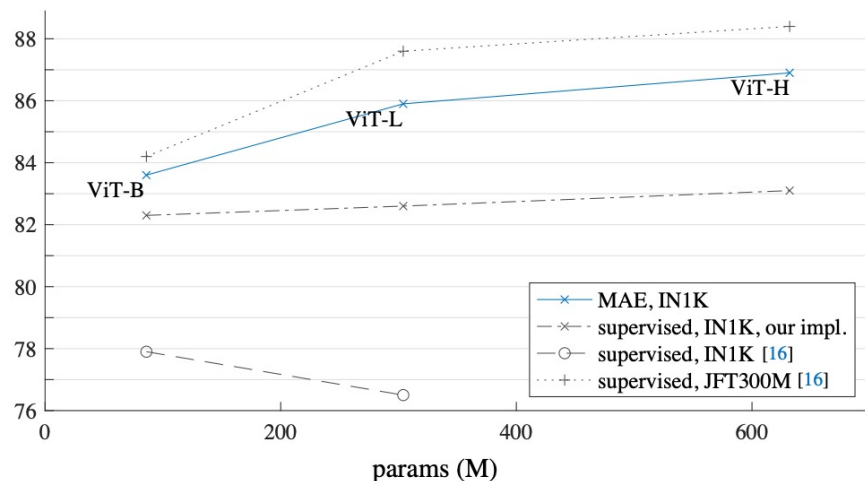


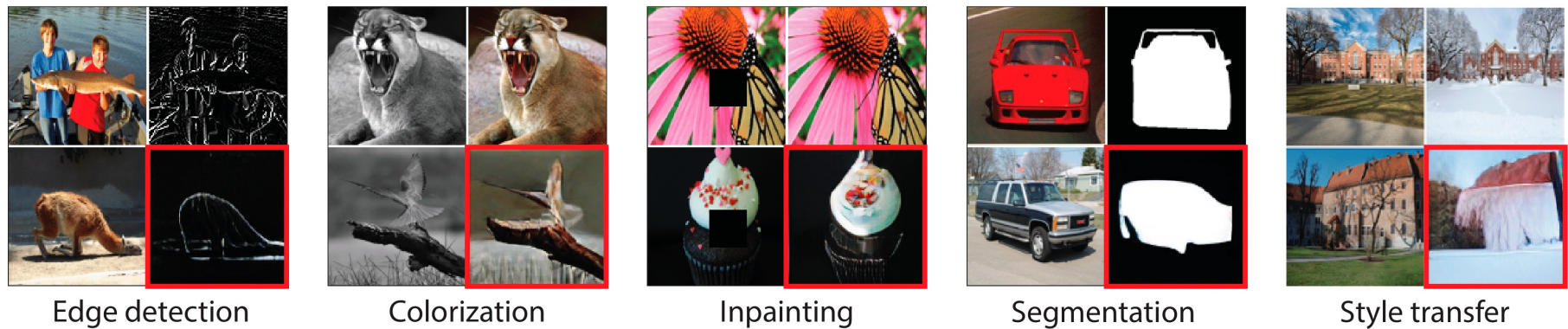
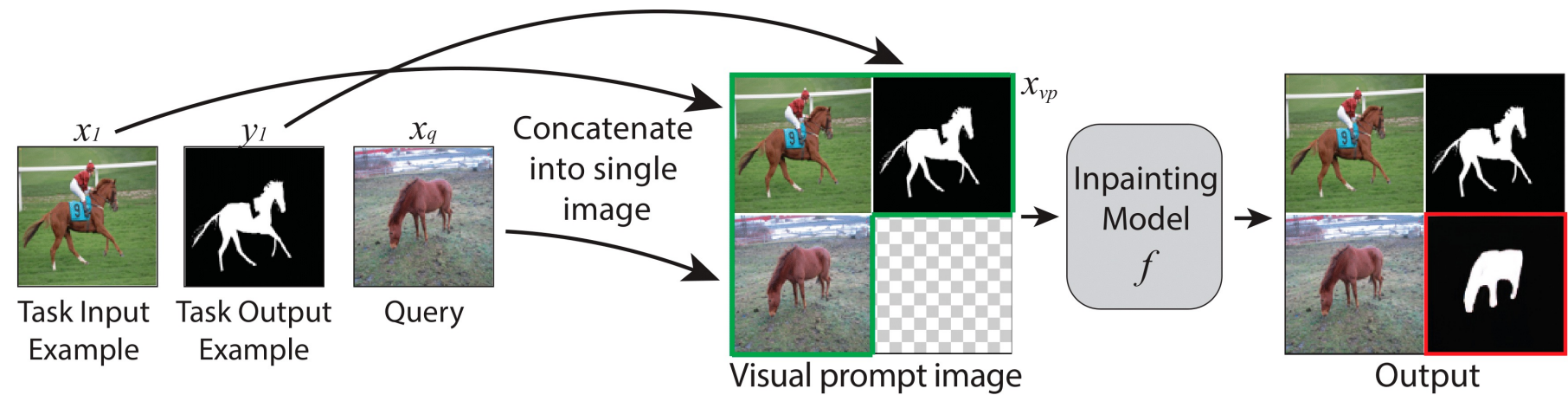
Figure 8. **MAE pre-training vs. supervised pre-training**, evaluated by fine-tuning in ImageNet-1K (224 size). We compare with the original ViT results [16] trained in IN1K or JFT300M.

method	pre-train data	ViT-B	ViT-L	ViT-H	ViT-H ₄₄₈
scratch, our impl.	-	82.3	82.6	83.1	-
DINO [5]	IN1K	82.8	-	-	-
MoCo v3 [9]	IN1K	83.2	84.1	-	-
BEiT [2]	IN1K+DALLE	83.2	85.2	-	-
MAE	IN1K	<u>83.6</u>	<u>85.9</u>	<u>86.9</u>	87.8

method	pre-train data	AP ^{box}		AP ^{mask}	
		ViT-B	ViT-L	ViT-B	ViT-L
supervised	IN1K w/ labels	47.9	49.3	42.9	43.9
MoCo v3	IN1K	47.9	49.3	42.7	44.0
BEiT	IN1K+DALLE	49.8	53.3	44.4	47.1
MAE	IN1K	50.3	53.3	44.9	47.2

Table 4. **COCO object detection and segmentation** using a ViT Mask R-CNN baseline. All entries are based on our implementation. Self-supervised entries use IN1K data *without* labels. Mask AP follows a similar trend as box AP.

Application of MAE: Visual prompting



A. Bar et al. [Visual prompting via image inpainting](#). NeurIPS 2022

Application of MAE: Visual prompting



Figure 3: **Random images from our Computer Vision Figures dataset.** We curated a dataset of 88k unlabeled figures from Computer Vision academic papers. During training, we randomly sample crops from these figures, without any additional parsing.

Application of MAE: Visual prompting

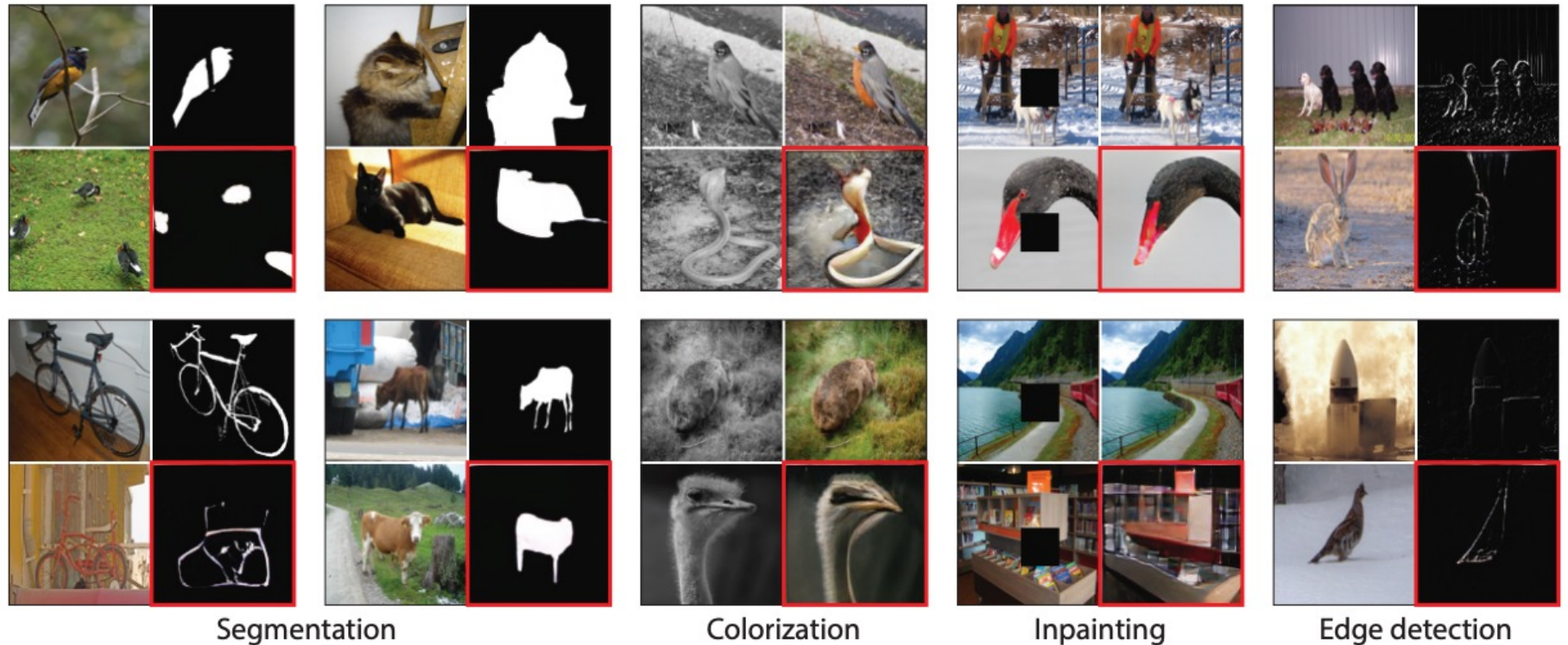


Figure 4: **Visual prompting prediction examples.** Each visual prompt was fed to an MAE-VQGAN model trained on the Figures dataset. For each visual prompt, the result is marked in red.

Application of MAE: Visual prompting

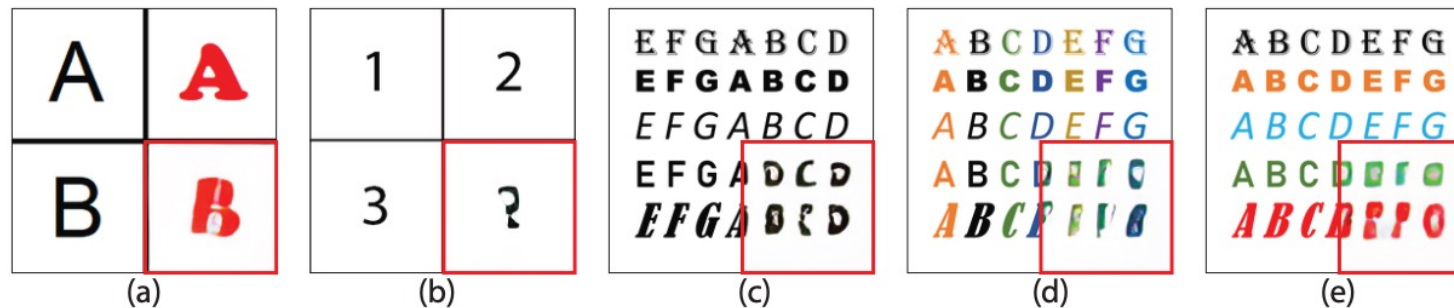


Figure 10: **Style and content extrapolation using MAE-VQGAN.** The model can extrapolate the style of a new content (a), but fails to predict a new content (b). The model struggles to extrapolate new style and content of longer sequences (c-e).

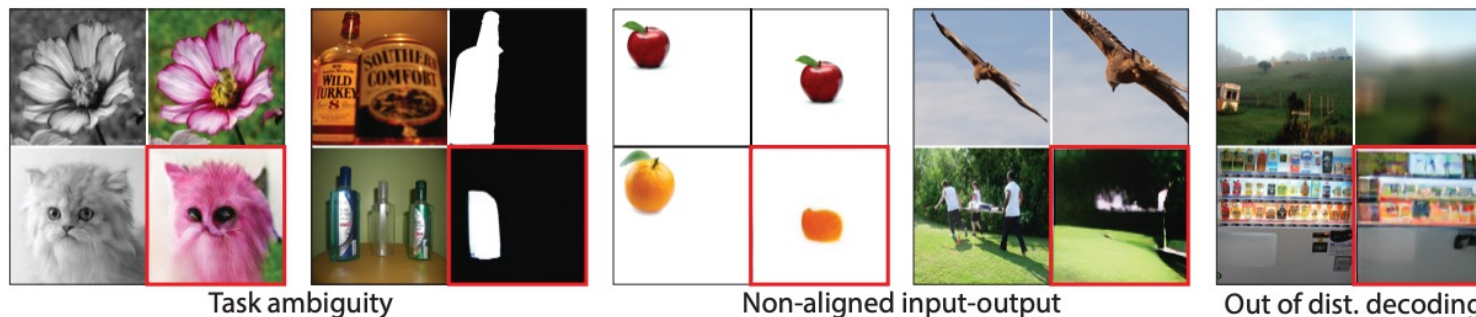
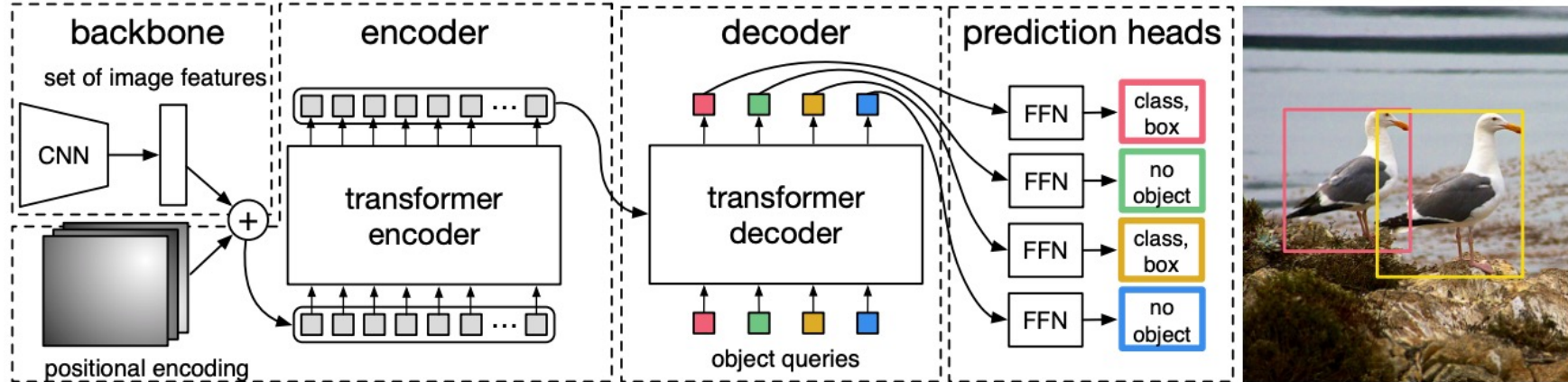
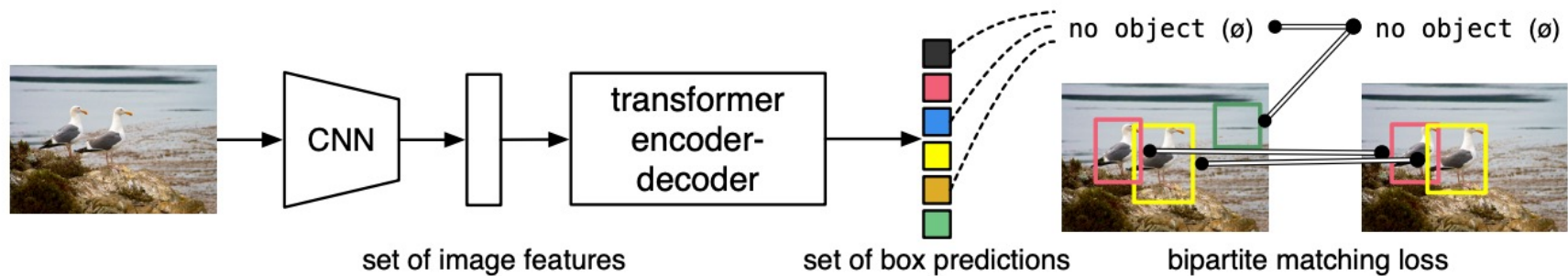


Figure 11: **Limitations and failure cases.** Single input-output example might be ambiguous and can lead to unintended completions. The MAE-VQGAN model performs worse given non-aligned input-output example, and by using a VQGAN vocabulary, it is limited in synthesizing out-of-distribution pixels (like blurry images).

Outline

- Transformer backbones for images: ViT, SWIN
- Self-supervised pre-training: DINO, MAE
- Dense prediction with transformers: Mask2Former, SAM

Detection Transformer (DETR)



N. Carion et al., [End-to-end object detection with transformers](#), ECCV 2020

MaskFormer

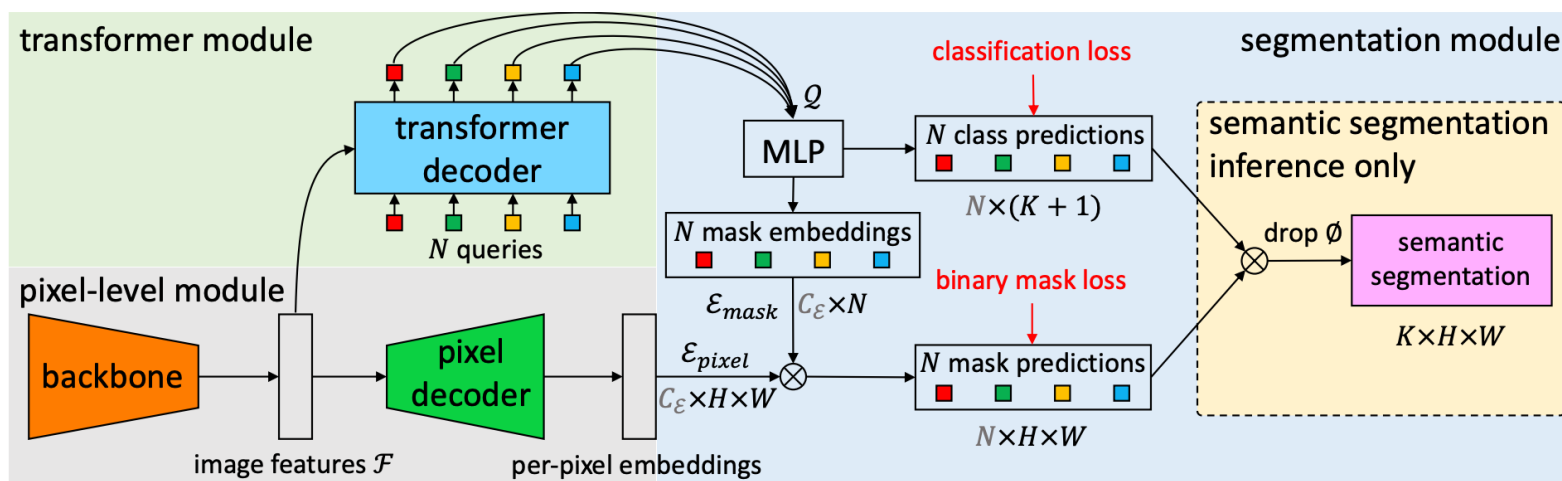
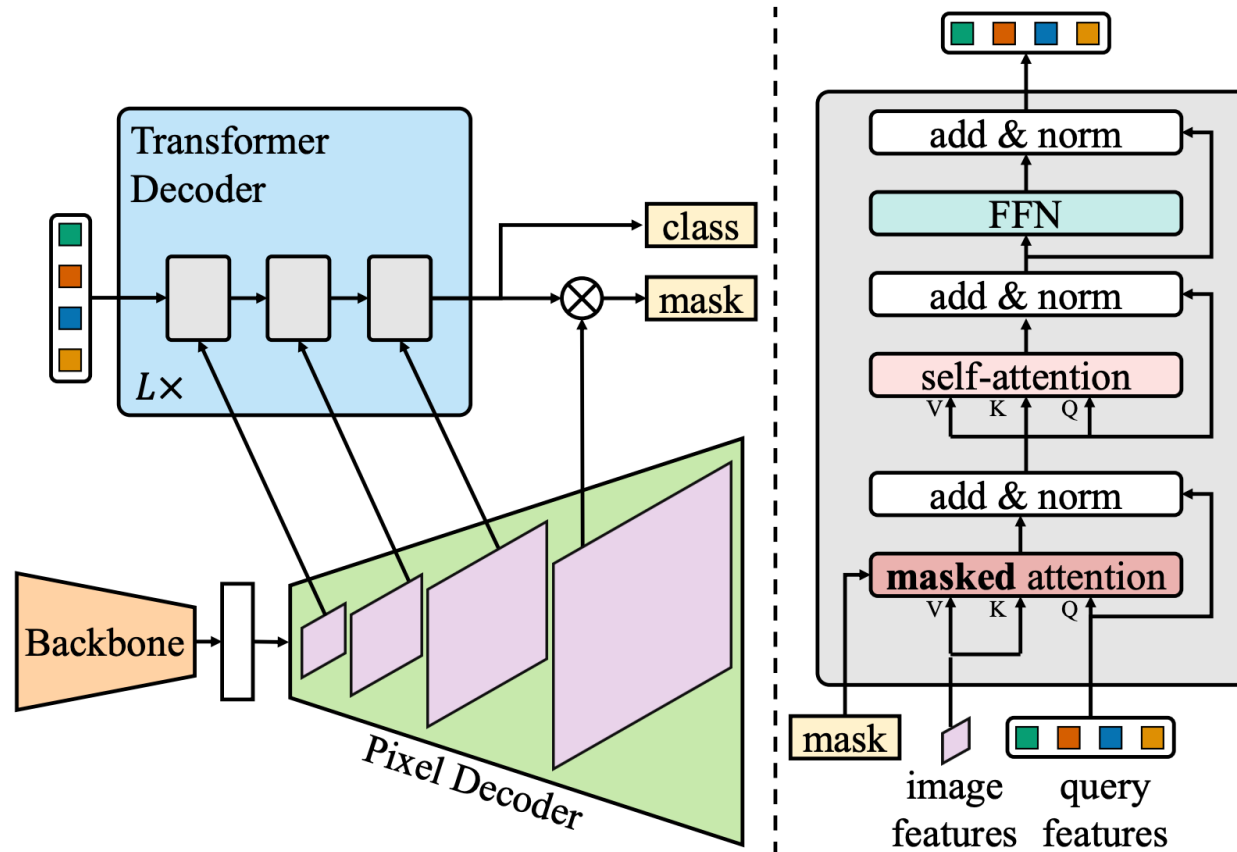


Figure 2: **MaskFormer overview.** We use a backbone to extract image features $\mathcal{F}$. A pixel decoder gradually upsamples image features to extract per-pixel embeddings $\mathcal{E}_{\text{pixel}}$. A transformer decoder attends to image features and produces N per-segment embeddings Q . The embeddings independently generate N class predictions with N corresponding mask embeddings $\mathcal{E}_{\text{mask}}$. Then, the model predicts N possibly overlapping binary mask predictions via a dot product between pixel embeddings $\mathcal{E}_{\text{pixel}}$ and mask embeddings $\mathcal{E}_{\text{mask}}$ followed by a sigmoid activation. For semantic segmentation task we can get the final prediction by combining N binary masks with their class predictions using a simple matrix multiplication (see Section 3.4). Note, the dimensions for multiplication $\otimes$ are shown in gray.

Mask2Former



B. Cheng et al. [Masked-attention Mask Transformer for Universal Image Segmentation](#). CVPR 2022

Segment Anything Model (SAM)

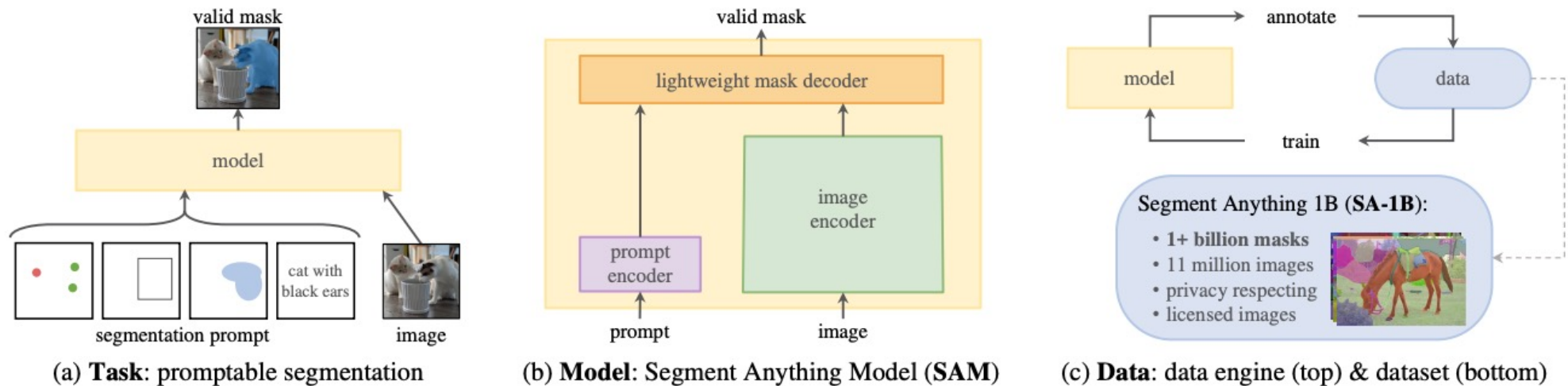


Figure 1: We aim to build a foundation model for segmentation by introducing three interconnected components: a promptable segmentation *task*, a segmentation *model* (SAM) that powers data annotation and enables zero-shot transfer to a range of tasks via prompt engineering, and a *data* engine for collecting SA-1B, our dataset of over 1 billion masks.

A. Kirillov et al. [Segment Anything](https://segment-anything.com/). ICCV 2023
<https://segment-anything.com/>

Segment Anything Model (SAM)

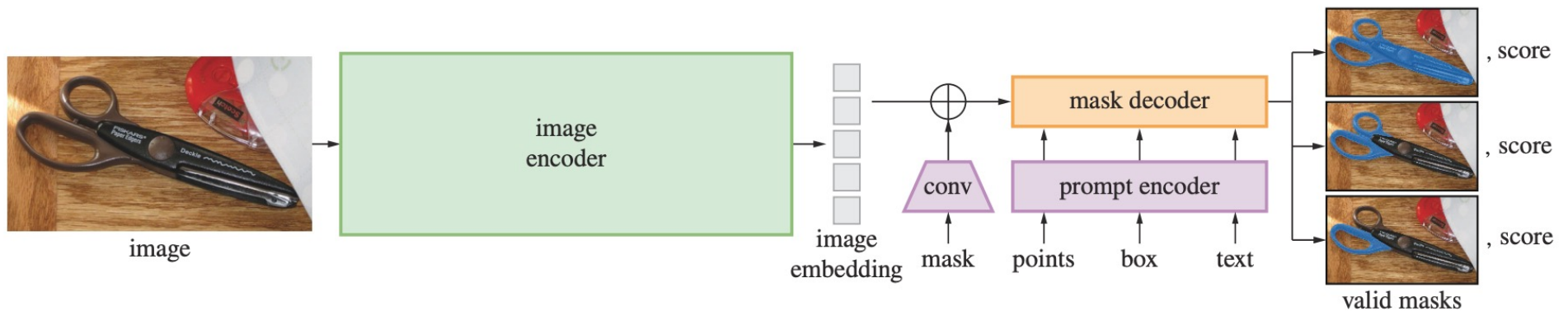


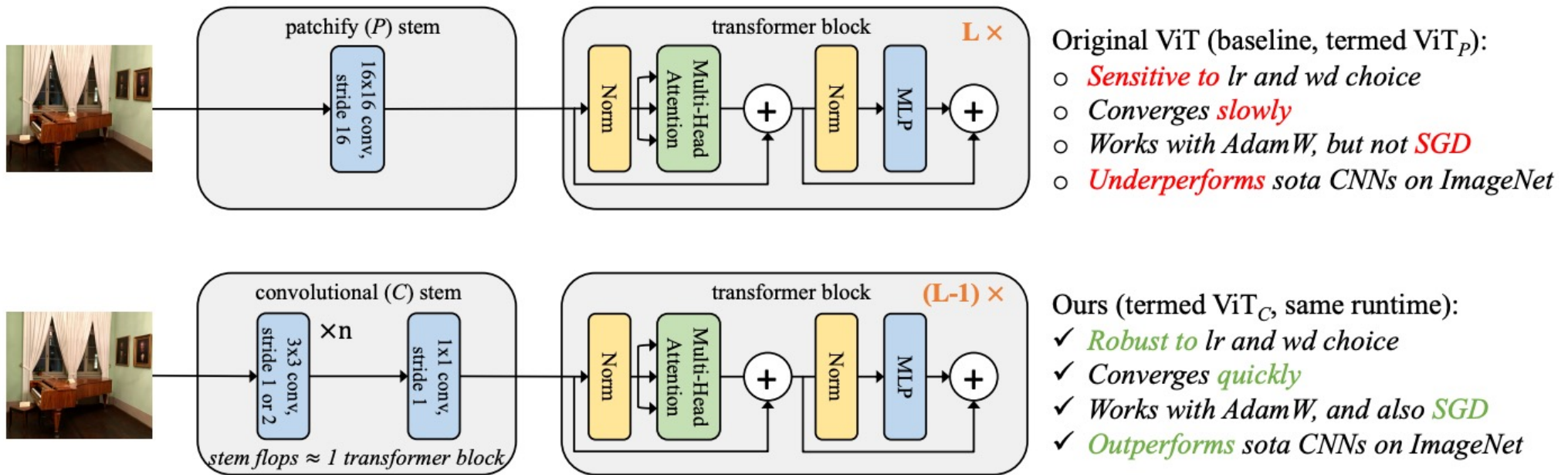
Figure 4: Segment Anything Model (SAM) overview. A heavyweight image encoder outputs an image embedding that can then be efficiently queried by a variety of input prompts to produce object masks at amortized real-time speed. For ambiguous prompts corresponding to more than one object, SAM can output multiple valid masks and associated confidence scores.

A. Kirillov et al. [Segment Anything](https://segment-anything.com/). ICCV 2023
<https://segment-anything.com/>

Outline

- Transformer backbones for images: ViT, SWIN
- Self-supervised pre-training: DINO, MAE
- Dense prediction with transformers: Mask2Former, SAM
- Alternatives to transformer architectures?

Hybrid of CNNs and transformers?



T. Xiao et al. [Early convolutions help transformers see better](#). NeurIPS 2021

Beyond transformers?

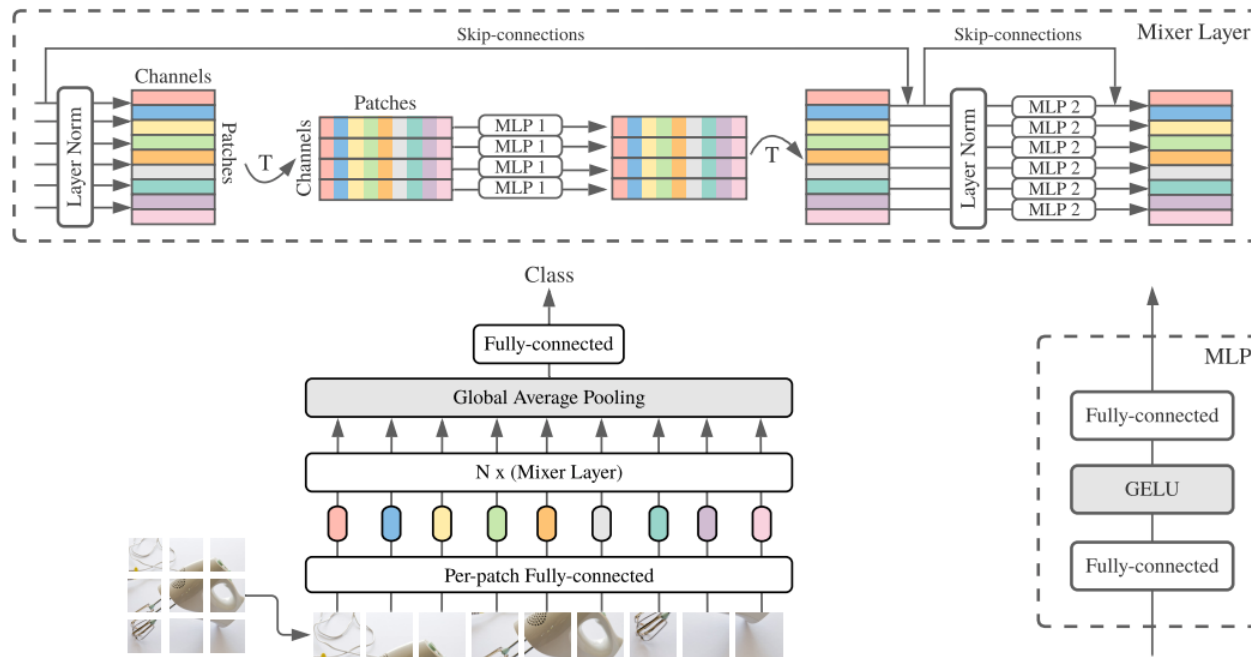


Figure 1: MLP-Mixer consists of per-patch linear embeddings, Mixer layers, and a classifier head. Mixer layers contain one token-mixing MLP and one channel-mixing MLP, each consisting of two fully-connected layers and a GELU nonlinearity. Other components include: skip-connections, dropout, and layer norm on the channels.

I. Tolstikhin et al. [MLP-Mixer: An all-MLP Architecture for Vision](#). NeurIPS 2021

Or completely back to CNNs?

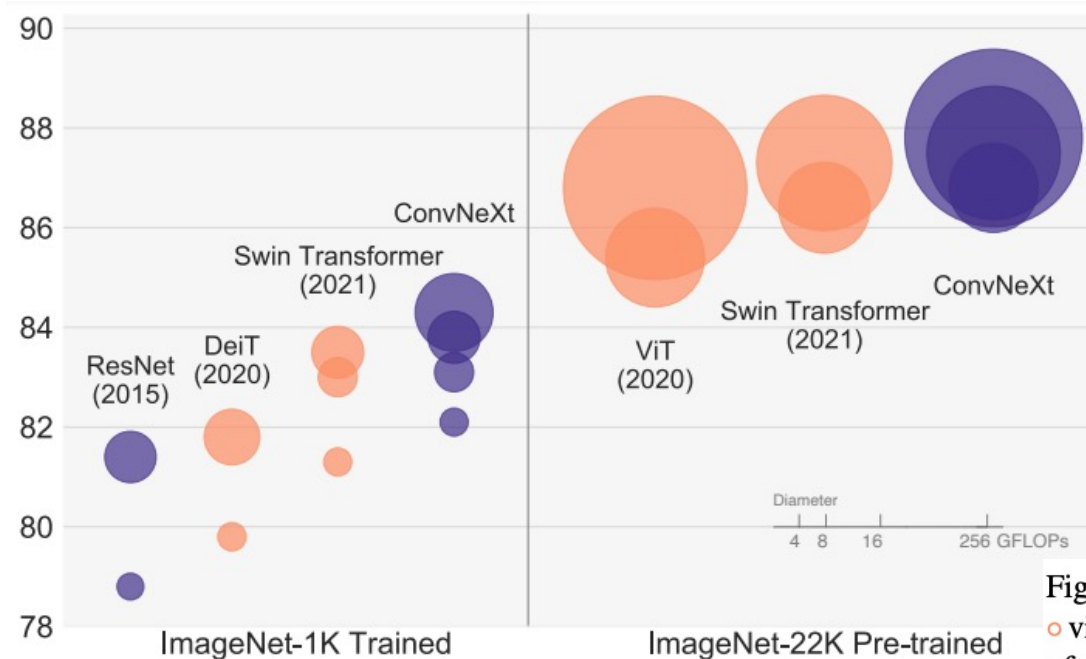


Figure 1. **ImageNet-1K classification** results for • ConvNets and • vision Transformers. Each bubble's area is proportional to FLOPs of a variant in a model family. ImageNet-1K/22K models here take $224^2/384^2$ images respectively. ResNet and ViT results were obtained with improved training procedures over the original papers. We demonstrate that a standard ConvNet model can achieve the same level of scalability as hierarchical vision Transformers while being much simpler in design.

Back to CNNs?

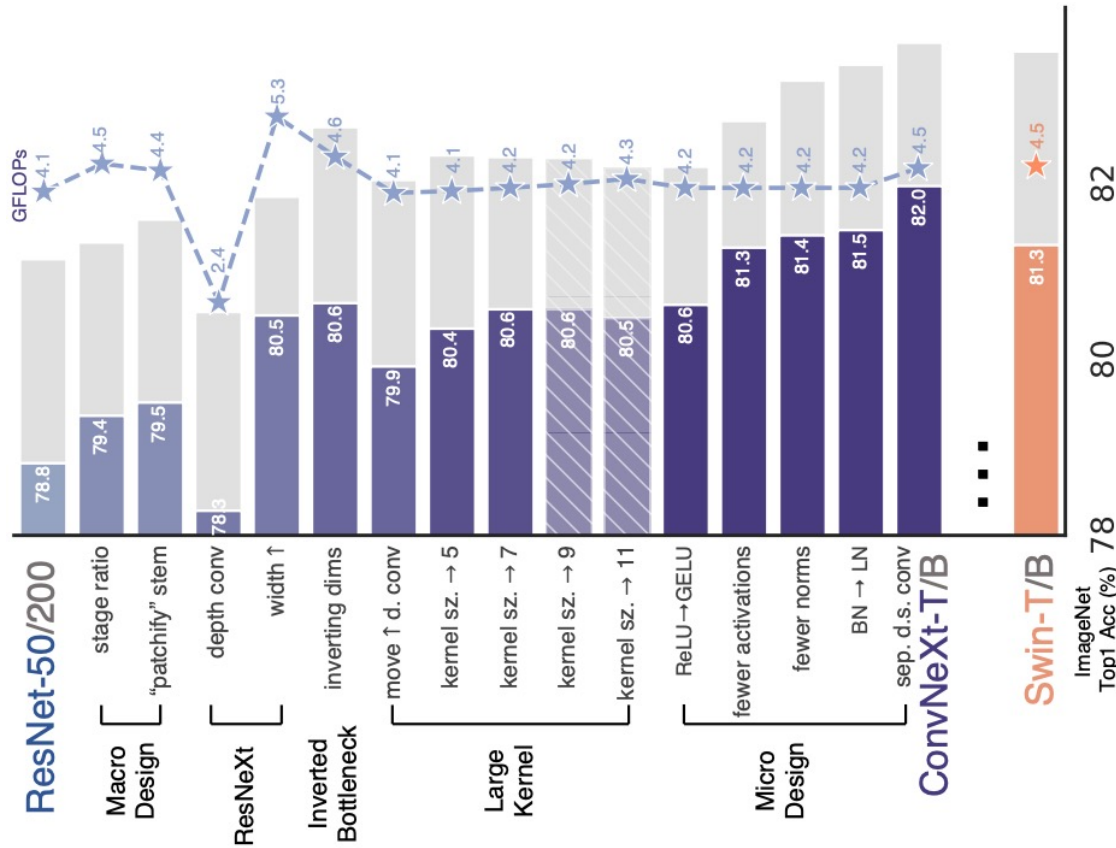
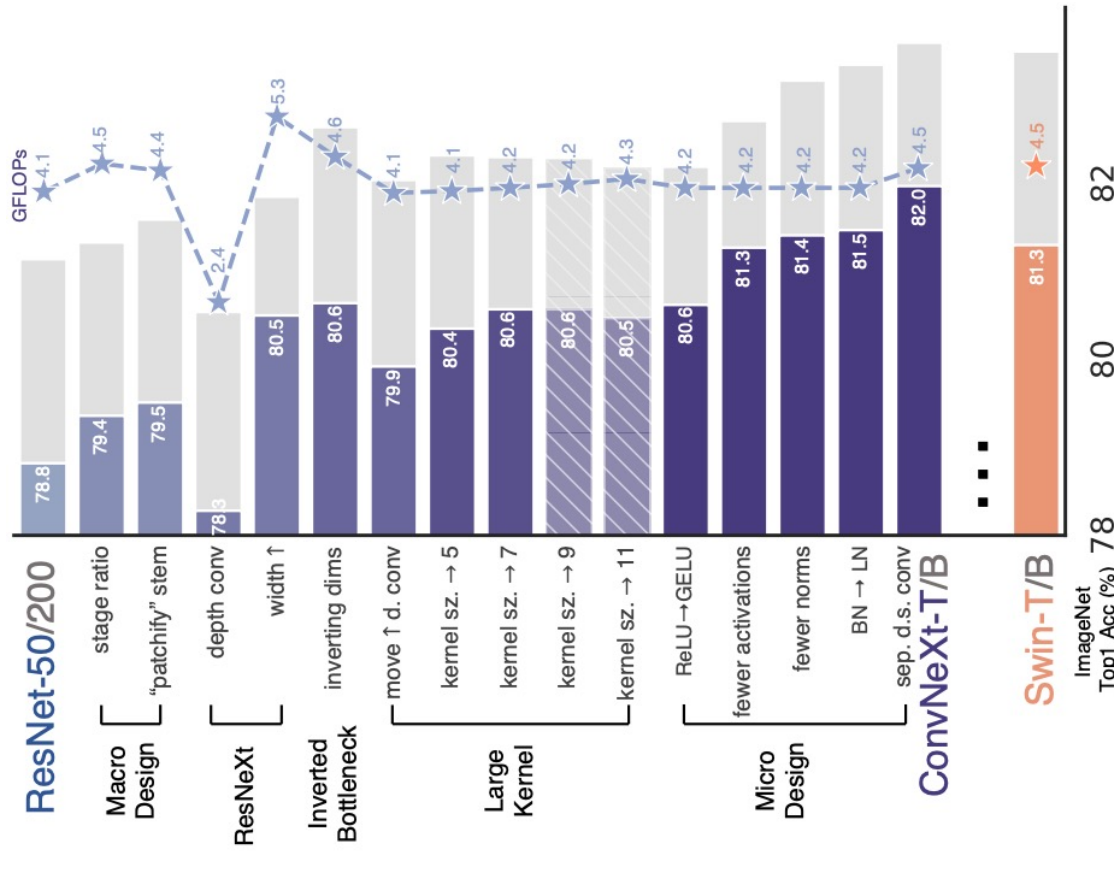
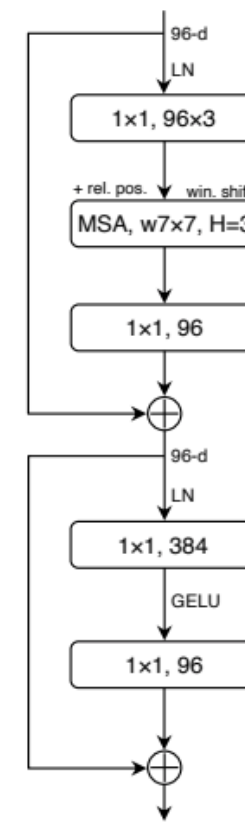


Figure 2. We modernize a standard ConvNet (ResNet) towards the design of a hierarchical vision Transformer (Swin), without introducing any attention-based modules. The foreground bars are model accuracies in the ResNet-50/Swin-T FLOP regime; results for the ResNet-200/Swin-B regime are shown with the gray bars. A hatched bar means the modification is not adopted. Detailed results for both regimes are in the appendix. Many Transformer architectural choices can be incorporated in a ConvNet, and they lead to increasingly better performance. In the end, our pure ConvNet model, named ConvNeXt, can outperform the Swin Transformer.

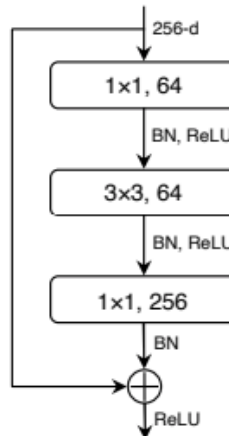
Back to CNNs?



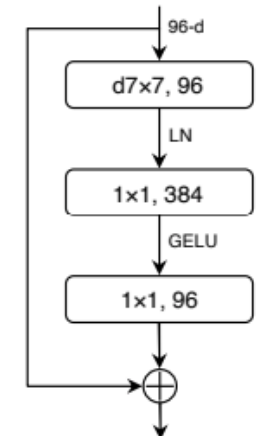
Swin Transformer Block



ResNet Block



ConvNeXt Block



Z. Liu et al. [A ConvNet for the 2020s](#). CVPR 2022

ConvNeXT v2

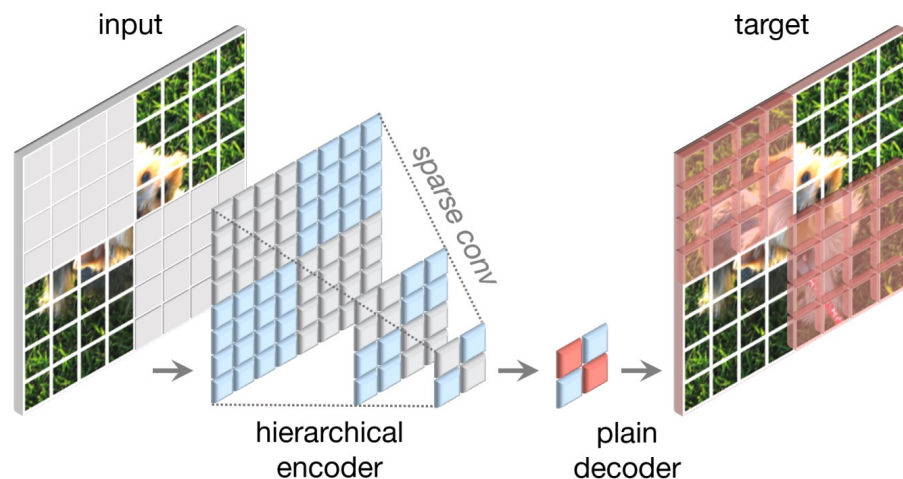


Figure 2. **Our FCMAE framework.** We introduce a fully convolutional masked autoencoder (FCMAE). It consists of a sparse convolution-based ConvNeXt encoder and a lightweight ConvNeXt block decoder. Overall, the architecture of our autoencoder is asymmetric. The encoder processes only the visible pixels, and the decoder reconstructs the image using the encoded pixels and mask tokens. The loss is calculated only on the masked region.

Backbone	Method	#param	PT epoch	FT acc.
ViT-B	BEiT	88M	800	83.2
ViT-B	MAE	88M	1600	83.6
Swin-B	SimMIM	88M	800	84.0
ConvNeXt V2-B	FCMAE	89M	800	84.6
ConvNeXt V2-B	FCMAE	89M	1600	84.9
ViT-L	BEiT	307M	800	85.2
ViT-L	MAE	307M	1600	<u>85.9</u>
Swin-L	SimMIM	197M	800	85.4
ConvNeXt V2-L	FCMAE	198M	800	85.6
ConvNeXt V2-L	FCMAE	198M	1600	85.8
ViT-H	MAE	632M	1600	<u>86.9</u>
Swin V2-H	SimMIM	658M	800	85.7
ConvNeXt V2-H	FCMAE	659M	800	85.8
ConvNeXt V2-H	FCMAE	659M	1600	86.3

Table 4. **Comparisons with previous masked image modeling approaches.** The pre-training data is the IN-1K training set. All self-supervised methods are benchmarked by the end-to-end fine-tuning performance with an image size of 224. We underline the highest accuracy for each model size and bold our best results.